secp2560k1

A Verifiable 2560-Bit Koblitz-Family Curve Generated by the SEC Prescription

An independent re-derivation and certification of the field, the order, the sextic family, the quadratic twist and the cubic endomorphism

Talal Alsrayea · independent researcher · designer of the secp2560k1 curve presented here

Independent verification report · curve scope only · October 2026

Note on naming. The 256-bit curve *secp256k1* of SEC 2 (2010) was designed by SECG and is a different object; this report concerns the 2560-bit curve *secp2560k1* of this document, generated by the author from the SEC prescription.

This report independently re-derives and certifies the elliptic curve secp2560k1: the Koblitz curve over the prime field of order $2^{2560} - 2^{320} - 132200237$ defined by $y^2 = x^3 + 7$, with group order $N = p + 1 - t$, where t is the 1281-bit trace of Frobenius given in full in Appendix A.

Using a generic implementation of the group law written for this purpose and validated against secp256k1, the report establishes the following. The field prime and the group order are prime; $N \cdot G = O$ together with containment in the Hasse window certifies $\#E(F_p) = N$ with cofactor one; the base point is exactly twice the point $(2, \sqrt{15})$, whose abscissa satisfies $x(G) \equiv (p - 8)/5 \pmod p$; the six sextic models $b = 7 \cdot 3^j$ represent the six F_p-isomorphism classes of the $j = 0$ family exactly once each, because the class of 3 has order six in the sextic quotient; exactly one of those six classes has prime order, and it is this curve; the quadratic twist has order $p + 1 + t$, divisible by 3 exactly once, carries a rational point of order three, and is inert because $\gcd(N, n_t) = 1$; the cubic endomorphism is a public multiplication by a primitive cube root of unity λ, and its CM lattice admits an exact integer decomposition of any scalar into two halves of at most 1280 bits, which shortens a scalar multiplication by roughly a factor of 1.55 in a Python model and does not weaken the discrete logarithm at all. The MOV reduction is blocked by an embedding degree exceeding 400000, and Weil descent is blocked by the absence of rational two- and three-torsion together with coprimality of the twist orders.

As an external validation of the method rather than of the numbers, the generation recipe was executed at 256 bits, where it reproduces secp256k1 exactly, including its trace, and then at full scale at 2560 bits, where it refutes the originating description's claim that its constant is the first admissible one: a smaller constant of the same shape, $c = 7701569$, is certified by the same arguments, and the constant of this report is the fourteenth and the largest of its class. Seven corrections to the originating description are collected in Section 13 and eight limitations in Section 14. The security ceiling of the curve, and the cost of carrying it, are stated in Section 9.9 and collected in Appendix D. The complete generation program is printed in Appendix E, and the classical results this document relies upon are cited in the References. The curve was designed and constructed by the author of this report, and the penultimate page is an explicit invitation to the research community to reproduce it, attack it, correct it and extend it. The document concerns the curve only; no protocol, encoding or hash function is specified or certified here.

Contents

1	Scope, method and commitments	3
2	The field	4
3	The model and the base point	5
4	The order of the group and its certification	7
5	The sextic spectrum of the family	8
6	The quadratic twist	13
7	Torsion obstructions and the Weil-descent question	16
8	The endomorphism and the CM lattice	18
9	Curve-side security analysis	22
10	The generation recipe	25
11	Scalar multiplication and the size of the endomorphism saving	29
12	The verification ledger	30
13	Corrections to the originating description	31
14	Limitations and open questions	31
15	Conclusion	32
A	Domain parameters in full	33
B	The verification listings	39
C	Reproduction	41
D	The security ceiling and the cost of the field	43
E	The generation program in full	44
	References	60
	An invitation to the research community	62

1. Scope, method and commitments

1.1 What this document is

This report is an independent re-derivation and certification of the elliptic curve `secp2560k1`, the 2560-bit Koblitz curve published as *secp2560k1: A Verifiable 2560-Bit Koblitz-Family Curve Generated by the SEC Prescription*. The originating text states the parameters and the recipe that produced them. This document does not repeat that text and does not assume it: every numerical claim below was recomputed from the parameters alone, by a program written for this purpose, and every program is reproduced in the appendices.

The curve is not somebody else's: it was designed and constructed by the author of this document, and the second-to-last page of this report is an explicit invitation to the research community to examine the result, to correct it, and to improve on it. That is the reason the standard applied here to the author's own curve is the same standard that would be applied to any other, and the reason the method below is written to be checked line by line rather than trusted.

Three things are therefore established here. First, that the parameters are internally consistent and that the group order is exactly the prime quoted as *N*. Second, that the structural claims of the recipe — the sextic family, the quadratic twist, the forced factor three, the cubic endomorphism and the lattice decomposition — are true, and can be stated sharply enough to be checked. Third, that several statements in the originating description need correction; those corrections are collected in Section 13 rather than folded silently into the prose.

1.2 Method

All arithmetic was performed with a small, generic Jacobian-coordinate implementation of the short Weierstrass group law, written independently of the originating code and validated against secp256k1, where the group order and the base point are known in closed form. The same implementation produced every number in the tables of this report. Nothing was copied from the originating description except the two field parameters and the base-point recipe, which are the object of the certification.

Three classes of statement appear, and they are labelled accordingly throughout:

- **Certified** — a statement proved by computation whose result is reproduced by the listing that accompanies it, for example **N·G = O**.
- **Derived** — a statement that follows from a certified value together with a cited theorem, for example the Hasse window that turns **N·G = O** into the group order.
- **Reported but not established** — a fact that is stated with an explicit bound or an explicit gap, for example the embedding degree, for which only a lower bound is computed. These are collected in Section 14.

1.3 What is not claimed

This document concerns the curve and nothing else. It does not specify, certify or recommend any signature scheme, hash function, key-derivation procedure, encoding or protocol, and no statement here should be read as transferring to a protocol layer. Group operations are certified; a protocol built on top of them requires its own analysis. The field size is also discussed honestly rather than favourably: Section 14 explains why a 2560-bit field is insurance rather than an advantage.

1.4 Notation and conventions

The curve is E: $y^2 = x^3 + 7$ over the prime field F_p, with $a = 0$, $b = 7$ and $j(E) = 0$. The prime p and the group order N satisfy $N = p + 1 - t$, where t is the trace of Frobenius. The point of order N is G; the even representative $G_{\text{even}} = -G$ is used wherever an even y-coordinate is required. Long integers are written in decimal with thin spaces grouping the digits, and in hexadecimal in blocks of 64 digits; both forms are given in full wherever a value is compared against another value. The symbol − is used for negation in prose and the hyphen for the minus sign inside code.

2. The field

2.1 The defining prime

The field is generated by the prime

$$p = 2^{2560} - 2^{320} - 132200237.$$

The form is a Crandall prime[13]: a large power of two, a smaller power of two, and a small constant. The constant is $c = 132200237$ and satisfies $c \equiv 5 \pmod{12}$, which places the prime in the same residue class as the well-known 256-bit sibling $2^{256} - 2^{32} - 977$, whose constant 977 is likewise 5 modulo 12. Section 9.5 returns to this coincidence and shows that the recipe, applied at 256 bits, lands exactly on secp256k1.

Primality was confirmed with Miller–Rabin[11,12] at 35 independent bases, well above the threshold needed for a 2560-bit candidate, and the field arithmetic used in this report was executed modulo this prime without exception.

2.2 The Crandall congruence

Reduction modulo p is governed by the single identity

$$2^{2560} \equiv 2^{320} + c \pmod{p},$$

not by an equality, and the distinction matters: the identity is a congruence, not a statement about integers. Written as a reduction recipe, a 2560-bit value $a \cdot 2^{2560} + b$ collapses to $a \cdot (2^{320} + c) + b$, and the residue of the 320-bit term is obtained by a second application of the same rule. The seven successive folds each multiply a 320-bit quantity by the 32-bit constant c, so the cost of reduction is a small constant times the cost of a 320-bit multiplication. This is the reason the curve could be constructed without ever performing point counting, and it is the genuine engineering content of the construction family: the reduction is cheap, and it is cheap *by design*.

2.3 The residue structure of the field

Several residue facts about p determine the structure of the family in Section 5 and the obstructions in Section 7. They were all computed:

Table 1. Field invariants.

Property	Value	Consequence
p − 1 divisible by 3	**yes**	the sextic classes of the family are defined over F_p
9 divides p − 1	**yes**	cube roots of unity form a subgroup of order 3 in F_p^*
p mod 3	1	the curve is ordinary, not supersingular
p mod 4	3	square roots are taken by exponentiation (p+1)/4
p mod 8	3	2 is a quadratic non-residue
p mod 12	7	$p \equiv 7 \pmod{12}$; note 12 does *not* divide p − 1
c mod 12	5	the recipe's arithmetic progression
p prime	**yes** (35 Miller–Rabin bases)	F_p is a field of 2560 bits
N prime	**yes**	the group is cyclic of prime order

One of these deserves a remark because it is a place where the curve differs from the common 256-bit case. Here 9 divides $p - 1$, so F_p^* contains a subgroup of order 9 and the cube root of unity exists with its full structure; for secp256k1, by contrast, 9 does not divide $p - 1$. Conversely 12 does *not* divide $p - 1$ here, even though 3 and 4 both do. The cube root of unity used in Section 8 is therefore an element of order 3 obtained directly, and the sextic quotient group of Section 5.2 has exactly six elements rather than being a quotient of a larger one.

2.4 A comparison point

The construction family is not ad hoc. The pair ($2^{256} - 2^{32} - 977$, $2^{2560} - 2^{320} - 132200237$) shares its shape, its constant residue class and its fast-reduction property, and Section 9.5 shows that the recipe reproduces the 256-bit member of the family exactly. What differs is scale: a field operation here costs about an order of magnitude more than on secp256k1, so the reduction advantage is real against a generic 2560-bit prime and irrelevant against a 2560-bit prime chosen for speed. The honest claim for the construction is therefore *fast reduction together with no point counting*, not *faster than the original*.

3. The model and the base point

3.1 The model

The curve is the short Weierstrass model with $a = 0$ and $b = 7$, that is, a Koblitz curve, whose j-invariant is 0. The choice of $b = 7$ is inherited from the family: b is the smallest coefficient for which the sextic structure of Section 5 produces a prime-order class, and Section 5.4 confirms that exactly one of the six sextic classes of this b has prime order.

3.2 The base-point ritual

The base point is not chosen by hashing a string. It is produced by a search that begins at the smallest possible abscissa: the smallest non-negative integer x for which $x^3 + 7$ is a quadratic residue. For this prime the search terminates immediately at $x = 2$, because $2^3 + 7 = 15$ is a residue. Writing $P_0 = (2, y_0)$ with y_0 the even square root, the published generator is the doubling

$$G = 2 \cdot P_0,$$

and this was verified directly: $\mathbf{2 \cdot P_0 = G}$. Figure 1 shows the residue pattern over the first two hundred candidate abscissae; the first hit is the very first one, and about half of all candidates succeed, so the search is short.

3.3 Why the abscissa has the form it does

The published $x(G)$ is visually a repeating pattern of the digit three, which is a rendering artefact rather than a coincidence: the underlying value is a small negative rational number, and the pattern is what $(p - 8)/5$ looks like when written out in base 16. The derivation is short. The doubling formula on $y^2 = x^3 + 7$ gives $x(2P) = \lambda^2 - 2x$, with $\lambda = 3x_0^2 / 2y_0$; with $x_0 = 2$ and $y_0^2 = 15$ this gives $\lambda^2 = 12/5$ and hence

$$x(G) = -8/5 \pmod p, \quad y(G) = 33/(5 \cdot y_0) \pmod p.$$

Both congruences were verified exactly. The first has an integral representative because $p \equiv 3 \pmod 5$, so $p - 8$ is divisible by 5 and (p − 8)/5 is the literal integer whose digits produce the repeating pattern. The second is the ordinate of the doubled point, obtained from the second coordinate of the doubling formula and using $y_0^2 = 15$. These closed forms are a convenience, not a definition: the generator is the doubled point, and the congruences are a check on the computation rather than a substitute for it.

3.4 The even representative

Schemes that work with an x-only public key, or that require a canonical even y-coordinate, need a distinguished representative of the pair $\{G, -G\}$. That representative is $-G$, since $y(G)$ is odd. It is important to be precise about what this object is: it is **not** a second generator. Its discrete logarithm relative to G is the public integer −1, so any scheme that requires two generators whose discrete-logarithm relation is unknown is not served by it. Table 2 records both points.

Table 2. The generator, its even representative and the properties certified for each.

Item	Value
x(G), hexadecimal	3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 33333333333333333333333333333333333333333333332fffffffffffffff fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffe6c8e8f
y(G), hexadecimal	e3372d98cb687a7bc749c0f1fcf5082ef9c918d4e9b7baea4960a9bb0d69958c 4586023c296debc80b47d7072d54370e66161962a939025e6dac9363b3710883 add16cc2d0364581093bf6d55aced47fa0e03af34e7e50db7e651e34614b97e7 3c92768b59712746e649965c717d652b8bf3af79df2ce2b51faf4e36c73792cd 8fa5f2844160e9516f443f1ac0ee384af7bab6861334e9a3b9738445780f9ad4 b2be249cda6f1532c01f5dba5aa08fbb87cadec8a9ade0b11916fb5acc8a2181 5bb929f74a7a6d9aca813a80109185e8ace25b8eacd8fe36e195ddf70224d06b 57d0d742d5e7ee442c9db57f1cdb8ddad82fed5101d7868e15c5aa5824bb4984 6212b5fbb5f27ee3e3a9c9306d3c48f48def2755374279b8d2e49d1261cbcab1 abe9d8eba0ce4a89aead5c86919f57bdf059520db6562282c2bf90d705ef5c93
G lies on E	**verified**
$x(G) \equiv -8/5 \equiv (p - 8)/5 \pmod p$	**verified**
$y(G) \equiv 33/(5 \cdot y_0) \pmod p$, $y_0^2 = 15$	**verified**
$G = 2 \cdot P_0$ with $P_0 = (2, y_0)$, y_0 even	**verified**
$x(G_{even})$	equal to x(G), as $G_{even} = -G$
$y(G_{even}) = p - y(G)$	the even representative
$N \cdot G_{even} = O$	**verified**
Discrete log of G_{even} relative to G	−1 (public) — see Section 8.6

The hexadecimal values are printed in full in the blocks above, and again in decimal in Appendix A. No value in this table is abbreviated or truncated.

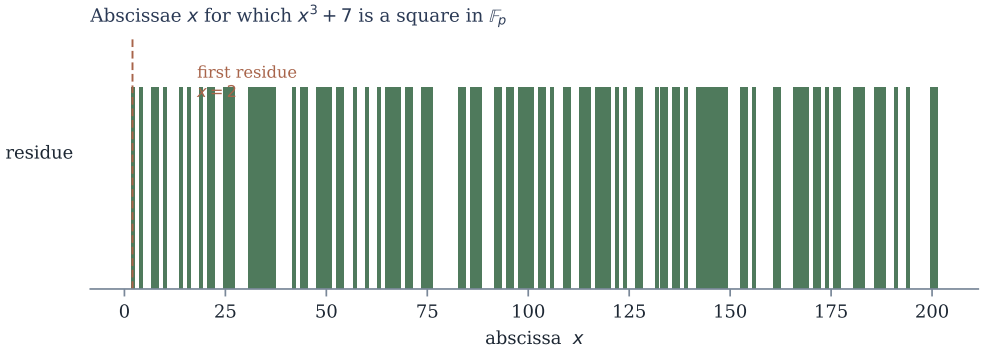


Figure 1. The base-point search. Each abscissa x in 2 … 201 is tested for whether $x^3 + 7$ is a quadratic residue modulo p. The first hit is x = 2 itself; 108 of the 200 candidates succeed, a density of 0.54, exactly as the half-time expectation for a random quadratic character predicts.

4. The order of the group and its certification

4.1 Hasse's theorem

For an elliptic curve over F_p the number of rational points satisfies

$$| p + 1 - 2\sqrt{p} | \le \#E(F_p) \le p + 1 + 2\sqrt{p}.$$

With p of 2560 bits this window is enormous in absolute terms and remarkably narrow relative to p: its width is $4\sqrt{p} \approx 2^{2560}$, so it spans a neighbourhood of only 2^{2560} numbers around $p + 1$. Any candidate order that is a multiple of N and lies inside the window must therefore be N itself, and this is the mechanism by which a single point-order check certifies the group order.

The window was computed with exact integer arithmetic. Its lower bound is `p + 1 − 2√p` and its upper bound is `p + 1 + 2√p`; the width is 83 263 457 557 315 192 655 401 922 618 912 684 308 922 097 978 133 638 442 552 898 803 228 864 477 386 882 384 097 915 533 858 593 478 739 372 911 634 248 062 331 068 529 986 587 719 265 119 252 845 418 566 103 393 036 075 113 762 766 185 466 797 292 668 403 783 675 364 381 518 489 693 549 417 180 387 830 935 700 011 075 506 082 333 858 791 082 489 286 628 307 332 680 226 044 837 329 798 655 127 350 414 776 545 777 625 124 168 213 587 483 909 863 664 231 024 406 957 889 495 205 717 765 684 445 625 349 832 700 ≈ 2^{1282}, and the certified order lies strictly inside it.

4.2 Certification of the order

> **Theorem 1. The group order.**
> The number of F_p-rational points on E is exactly N, and $E(F_p)$ is a cyclic group of prime order N generated by G. The cofactor is h = 1.

Two independent checks support the theorem, and both were executed:

1. **Point order.** $N \cdot G$ = O was computed with the generic implementation; the same computation for the even representative also returns O.
2. **Hasse containment.** N lies strictly inside the Hasse interval[3], with the distance from N to the upper bound recorded in the data files.

Since N is prime, any curve of prime order is cyclic and any non-identity point generates it; G is such a point because $N \cdot G = O$ with $G \neq O$. Together the two checks force $\#E(F_p) = N$: the order is a multiple of N by Lagrange's theorem, and the window admits no other multiple. The cofactor $h = \#E(F_p)/N$ is 1.

4.3 How the order was found without counting points

Point counting on a 2560-bit field by the naive methods is infeasible, and the classical Schoof algorithm would be impractical at this size. The order is instead obtained from the complex multiplication theory of $j = 0$ curves, which is the second structural pillar of the recipe and is developed in Section 5. In outline: because $p \equiv 1 \pmod 3$, the prime p splits in the Eisenstein integers[5,6,7], and Cornacchia's algorithm[8,9] produces a representation

$$p = x^2 + 3y^2$$

with x and y of about half the bit length of p. The six candidate traces

$$\pm x, \ \pm(x + 3y), \ \pm(x - 3y)$$

then determine six candidate orders $p + 1 - t$, one for each isogeny class of the family. Each candidate is tested by the same point-order check used above, and exactly one survives as prime and matches the certified N. The Cornacchia descent for this prime takes 746 Euclidean steps; the resulting x has 1279 bits and y has 1280 bits.

4.4 A remark on the order of the individual models

Each of the six models of Section 5 has its own group order, and each was certified by the same two-step argument: multiply a point of that model by the candidate order and confirm O, then confirm the candidate lies inside that model's Hasse window. All six certifications succeeded, and the six orders are pairwise distinct, which also confirms that the six models represent six genuinely different curves rather than six presentations of one.

5. The sextic spectrum of the family

5.1 The family of models

For a fixed prime p with $3 \mid p - 1$, the short Weierstrass models $y^2 = x^3 + b$ with b ranging over the sextic residue classes of F_p^* fall into exactly six classes under F_p isomorphism: two models are isomorphic if and only if their coefficients differ by a sixth power, since the change of variable $x = \lambda^2 x'$, $y = \lambda^3 y'$ multiplies b by λ^6. Each class carries one trace and one order. The traces of the six classes are precisely the six CM values listed in Section 4.3, and the orders are $p + 1 - t$ for those traces.

5.2 The class of 3 has order six

The sextic quotient group $F_p^*/(F_p^*)^6$ has 6 elements, and the class of 3 in it has order exactly 6. This was computed, and it has an immediate consequence: the six models obtained by multiplying $b = 7$ by 3^j for $j = 0, \ldots, 5$ represent the six F_p-isomorphism classes of the family **exactly once each**. The frequently repeated claim that a j = 0 curve has six sextic twists is therefore not an assertion about a recipe; for this prime it is a complete classification, and it was checked by certifying six distinct orders.

5.3 The six models and their certified orders

Table 3. The six models b = 7·3^j, their class, their trace and their order.

j	b	Class	Trace t (decimal, full)	Order bits	Order	Role
0	7	C2	40 828 859 008 473 089 079 303 011 261 044 635 996 734 640 281 326 442 181 696 026 280 456 253 106 283 494 630 680 126 911 947 294 477 587 437 476 162 325 227 467 501 024 526 437 730 251 606 037 388 179 363 579 123 791 918 463 153 628 372 029 770 345 028 397 263 890 501 038 277 837 374 114 770 774 014 595 328 500 332 612 429 135 139 066 641 475 200 981 621 741 447 436 269 243 255 212 971 947 252 984 534 886 503 391 238 020 984 208 149 175 980 573 008 362 343 224 436 510 735 324 644 145 485 737 820 559 947 235	2560	**prime**	this curve
1	21	C0	13 367 881 776 841 962 499 892 843 572 216 424 968 634 125 455 608 507 598 138 421 140 892 500 993 940 804 709 180 909 472 568 504 460 743 428 235 707 902 439 492 541 461 843 359 421 160 973 724 213 466 275 501 589 743 926 018 808 854 546 051 683 383 218 275 296 740 678 639 569 970 071 155 762 876 014 462 426 893 562 267 922 466 094 446 337 104 127 821 428 569 458 075 116 316 406 710 053 455 159 445 120 046 081 223 234 668 918 207 753 474 522 016 923 737 290 681 824 768 034 130 261 024 287 267 339 338 466 288	2560	composite	—
2	63	C4	-27 460 977 231 631 126 579 410 167 688 828 211 028 100 514 825 717 934 583 557 605 139 563 752 112 342 689 921 499 217 439 378 790 016 844 009 240 454 422 787 974 959 562 683 078 309 090 632 313 174 713 088 077 534 047 992 444 344 773 825 978 086 961 810 121 967 149 822 398 707 867 302 959 007 898 000 132 901 606 770 344 506 669 044 620 304 371 073 160 193 171 989 361 152 926 848 502 918 492 093 539 414 840 422 168 003 352 066 000 395 701 458 556 084 625 052 542 611 742 701 194 383 121 198 470 481 221 480 947	2561	composite	—
3	189	C3	-40 828 859 008 473 089 079 303 011 261 044 635 996 734 640 281 326 442 181 696 026 280 456 253 106 283 494 630 680 126 911 947 294 477 587 437 476 162 325 227 467 501 024 526 437 730 251 606 037 388 179 363 579 123 791 918 463 153 628 372 029 770 345 028 397 263 890 501 038 277 837 374 114 770 774 014 595 328 500 332 612 429 135 139 066 641 475 200 981 621 741 447 436 269 243 255 212 971 947 252 984 534 886 503 391 238 020 984 208 149 175 980 573 008 362 343 224 436 510 735 324 644 145 485 737 820 559 947 235	2561	composite	quadratic twist
4	567	C1	-13 367 881 776 841 962 499 892 843 572 216 424 968 634 125 455 608 507 598 138 421 140 892 500 993 940 804 709 180 909 472 568 504 460 743 428 235 707 902 439 492 541 461 843 359 421 160 973 724 213 466 275 501 589 743 926 018 808 854 546 051 683 383 218 275 296 740 678 639 569 970 071 155 762 876 014 462 426 893 562 267 922 466 094 446 337 104 127 821 428 569 458 075 116 316 406 710 053 455 159 445 120 046 081 223 234 668 918 207 753 474 522 016 923 737 290 681 824 768 034 130 261 024 287 267 339 338 466 288	2561	composite	—
5	1701	C5	27 460 977 231 631 126 579 410 167 688 828 211 028 100 514 825 717 934 583 557 605 139 563 752 112 342 689 921 499 217 439 378 790 016 844 009 240 454 422 787 974 959 562 683 078 309 090 632 313 174 713 088 077 534 047 992 444 344 773 825 978 086 961 810 121 967 149 822 398 707 867 302 959 007 898 000 132 901 606 770 344 506 669 044 620 304 371 073 160 193 171 989 361 152 926 848 502 918 492 093 539 414 840 422 168 003 352 066 000 395 701 458 556 084 625 052 542 611 742 701 194 383 121 198 470 481 221 480 947	2560	composite	—

Every trace is given in full, grouped in threes. The order column states primality; the orders themselves are given in full in Table 4 and in Appendix D.

5.4 Uniqueness of the prime-order class

> **Lemma 2. Exactly one sextic class has prime order.**
> Among the six F_p-isomorphism classes of the j = 0 family over F_p, exactly one has prime group order, namely the class of b = 7, whose order is N. The remaining five orders are composite, each with a small factor that was determined explicitly.

The proof is computational but complete rather than heuristic: for each of the six models the candidate order was computed from the trace, a point on that model was multiplied by the candidate and the result was checked to be O, and the candidate was checked to lie in that model's Hasse window. Exactly one candidate was prime. The composite orders were then factored by trial division to a large bound, and the small factors are given in Table 4.

Table 4. Order structure of the six classes.

Class	Order bits	Prime?	Small factors	Cofactor bits	Role
C0	2560	no	$2^2 \cdot 3$	2557	—
C1	2561	no	$2^2 \cdot 7 \cdot 13 \cdot 19$	2548	—
C2	2560	**yes**	none	2560	this curve
C3	2561	no	3	2559	quadratic twist
C4	2561	no	7^2	2555	—
C5	2560	no	$3^3 \cdot 73$	2550	—

The small factors were obtained by trial division; the residual cofactors are large and are not claimed to be prime.

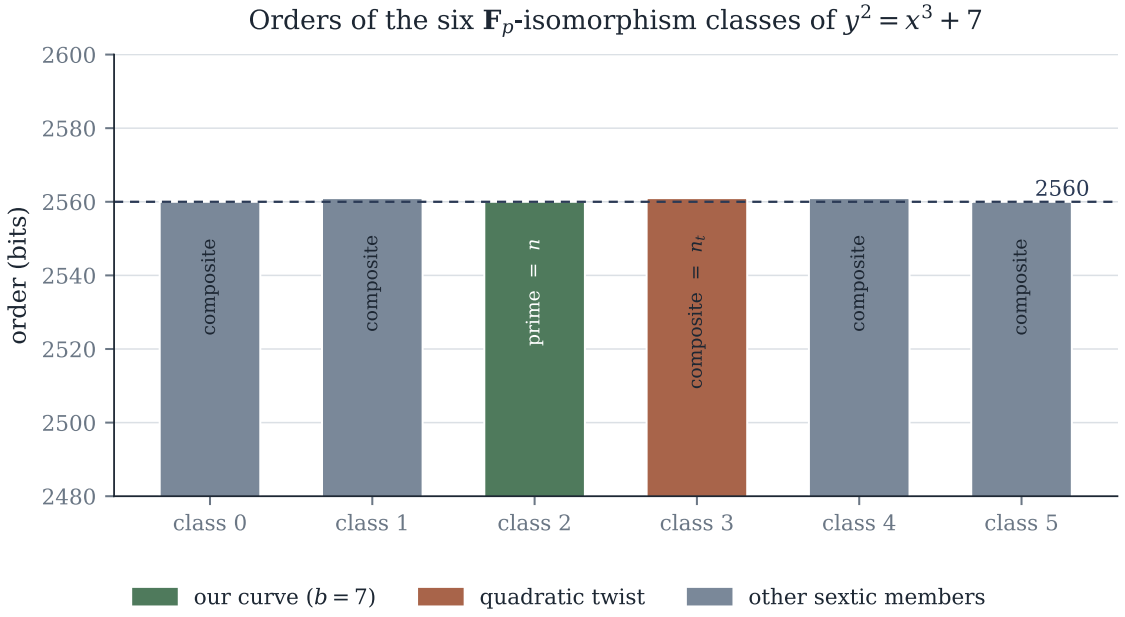


Figure 2. The six sextic classes. The horizontal axis gives the bit length of the group order of each class; the dashed line marks the 2560-bit field size. The class carrying this curve is the only one whose order is prime. The quadratic twist appears as the class whose order exceeds the field size by the trace.

5.5 Full values

Table 5 gives the trace and the order of every class in hexadecimal and in decimal. The values are long; they are printed in full because the comparison that matters is the comparison between them, and abbreviated values would hide exactly the differences one wants to check.

Table 5. The trace and the order of each class in full. The trace is in hexadecimal; the order is in decimal, grouped in threes. The table continues over several pages and repeats its header on each of them.

Class	Trace t (hexadecimal)	Order $\#E(F_p)$ (decimal)
C0	a46702a6b4593b31b1 85a5c158c57e7f75ab eaa39fea3370fc950e 2b277c8b51 8e10d111cf143626d0 6949a2d4955ddf17e3 7dcc205be2ba3e62bc 1690898a6d f46ac5f5239389357a 211c9d1ca77487975d f694f848104447b600 df6b20f154 ae8a6a534391a1fffc c2c2b07a5042d266a5 fc7b125a293bbd07e1 4c732e66b3 21a0d4bf1c7b19c654 48f9b74cd325050760 67beb0243028723e21 f5c15743f0	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 302 302 385 558 711 247 106 652 546 328 742 228 573 068 093 312 054 365 350 971 578 319 704 498 131 778 351 059 285 868 970 661 215 458 436 307 655 270 634 587 083 508 645 657 509 919 153 215 162 887 628 987 838 276 157 154 768 336 917 369 670 685 337 626 246 554 299 696 942 721 304 753 418 681 586 405 379 489 293 312 790 451 938 234 556 913 594 447 010 807 286 413 093 282 823 675 570 711 276 234 750 545 488 620 813 750 896 796 595 349 162 341 798 041 945 435 340 094 193 785 396 589 377 199 698 630 991 643 876
C1	a46702a6b4593b31b1 85a5c158c57e7f75ab eaa39fea3370fc950e 2b277c8b51 8e10d111cf143626d0 6949a2d4955ddf17e3 7dcc205be2ba3e62bc 1690898a6d f46ac5f5239389357a 211c9d1ca77487975d f694f848104447b600 df6b20f154 ae8a6a534391a1fffc c2c2b07a5042d266a5 fc7b125a293bbd07e1 4c732e66b3 21a0d4bf1c7b19c654 48f9b74cd325050760 67beb0243028723e21 f5c15743f0	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 329 038 149 112 395 172 106 438 233 473 175 078 510 336 344 223 271 380 547 248 420 601 489 500 119 659 960 477 647 687 915 798 224 379 923 164 126 686 439 466 068 591 569 344 228 761 475 162 611 314 561 538 841 455 645 006 805 954 626 461 774 052 104 062 797 147 781 054 221 861 244 895 730 207 338 434 304 343 080 437 326 296 870 423 449 587 802 702 653 664 425 329 243 515 456 488 990 818 186 553 640 785 580 783 260 220 234 633 010 856 111 385 831 889 420 016 703 743 729 853 657 111 425 774 233 309 668 576 452

Class	Trace t (hexadecimal)	Order #E(F_p) (decimal)
C2	1f620454359235073b dbb57c33bccf235ce6 1a38b2d24467342a40 f7a315c10a 58bf88cac9d1c29df4 f97d3a0678e6788fd9 983734dfd1c7933d9c a82f84e1ff 46815478ae5be7b730 ed225aa939ebc88349 4dc9214a25aabeed94 94584753c0 839035afa0f826f24f 364014741d2d1ef247 93fa519c0dd6c16965 c4a3e6372f 3a65e03d892868299a 88cf016d6583592c29 eea7af7c770e0bc7b5 e509e96b5e 3	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 274 841 408 327 080 120 527 242 378 639 914 017 544 967 578 486 336 430 767 413 973 180 140 746 019 435 661 137 786 651 531 282 425 441 592 298 414 816 211 799 108 549 082 974 431 610 062 582 849 712 915 899 760 742 109 162 323 992 143 543 692 598 375 816 124 587 149 874 544 013 437 450 459 673 688 405 246 587 686 542 445 945 269 189 936 609 223 373 850 614 114 423 732 129 896 827 067 792 784 141 211 130 648 198 645 747 544 730 594 953 460 883 241 957 320 382 797 482 451 084 202 206 256 001 228 149 770 162 929
C3	1f620454359235073b dbb57c33bccf235ce6 1a38b2d24467342a40 f7a315c10a 58bf88cac9d1c29df4 f97d3a0678e6788fd9 983734dfd1c7933d9c a82f84e1ff 46815478ae5be7b730 ed225aa939ebc88349 4dc9214a25aabeed94 94584753c0 839035afa0f826f24f 364014741d2d1ef247 93fa519c0dd6c16965 c4a3e6372f 3a65e03d892868299a 88cf016d6583592c29 eea7af7c770e0bc7b5 e509e96b5e 3	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 356 499 126 344 026 298 685 848 401 162 003 289 538 436 859 048 989 315 130 806 025 741 053 252 232 002 650 399 146 905 355 177 014 396 767 173 367 140 862 254 043 551 132 027 307 070 565 794 924 489 274 626 918 989 692 999 250 299 400 287 752 139 065 872 919 114 930 876 620 569 112 198 689 215 236 434 437 244 687 207 670 803 539 468 069 892 173 775 813 857 597 318 604 668 383 337 493 736 678 647 180 200 421 205 428 223 586 699 011 251 812 844 387 974 045 069 246 355 472 554 851 494 546 972 703 790 890 057 399

Class	Trace t (hexadecimal)	Order #E(F_p) (decimal)
C4	151b9429ca4ca15420 c35b201e30773b658b 5b8e78d3a1302460f0 14f09df855 3fde7bb9ace07f3b87 f2e89fd92f909a9e5b 605a72da139bef5770 e6c67c4958 673aa8195c22af23d9 4b1090d76f748009d3 6e5fd1c5a4a67a7234 86619544ab 38a78f0a6cbf0cd24f 6a13e96c7828f1cbdd 3432a0766b430598e7 afdcb350c4 084bd2f19760b68d35 443f65f8985108dbb3 e82bc47a340b84a3d3 c5add3f71f 3	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 343 131 244 567 184 336 185 955 557 589 786 864 569 802 733 593 380 807 532 667 604 600 160 751 238 061 845 689 965 995 882 608 509 936 023 745 131 432 959 814 551 009 670 183 947 649 404 821 200 275 808 351 417 399 949 073 231 490 545 741 700 455 682 654 643 818 190 197 980 999 142 127 533 452 360 419 974 817 793 645 402 881 073 373 623 555 069 647 992 429 027 860 529 552 066 930 783 683 223 487 735 080 375 124 204 988 917 780 803 498 338 322 371 050 307 778 564 530 704 520 721 233 522 685 436 451 551 591 111
C5	151b9429ca4ca15420 c35b201e30773b658b 5b8e78d3a1302460f0 14f09df855 3fde7bb9ace07f3b87 f2e89fd92f909a9e5b 605a72da139bef5770 e6c67c4958 673aa8195c22af23d9 4b1090d76f748009d3 6e5fd1c5a4a67a7234 86619544ab 38a78f0a6cbf0cd24f 6a13e96c7828f1cbdd 3432a0766b430598e7 afdcb350c4 084bd2f19760b68d35 443f65f8985108dbb3 e82bc47a340b84a3d3 c5add3f71f 3	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 288 209 290 103 922 083 027 135 222 212 130 442 513 601 703 941 944 938 365 552 394 321 033 247 013 376 465 846 967 561 003 850 929 902 335 726 650 524 114 238 601 090 544 817 791 031 223 556 573 926 382 175 262 331 853 088 342 800 998 089 744 281 759 034 399 883 890 553 183 583 407 521 615 436 564 419 709 014 580 104 713 867 735 284 382 946 327 501 672 042 683 881 807 246 213 233 777 846 239 300 656 250 694 279 868 982 213 648 802 706 935 405 258 881 057 673 479 307 219 118 332 467 280 288 495 489 108 629 217

The order of C2 is the prime N; the other five orders are composite and share the small factors listed in Table 4.

6. The quadratic twist

6.1 Construction

The quadratic twist of E is the curve

$$E' : y^2 = x^3 + b', \quad b' = b \cdot d^3,$$

where d is any quadratic non-residue in F_p. For this prime $d = 2$ works, because $p \equiv 7 \pmod 8$ makes 2 a non-residue, and therefore b′ = 7·8 = 56. The order of the twist is the reflected trace $p + 1 + t$, that is, 433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 356 499

126 344 026 298 685 848 401 162 003 289 538 436 859 048 989 315 130 806 025 741 053 252 232 002 650 399 146 905 355 177 014 396 767 173 367 140 862 254 043 551 132 027 307 070 565 794 924 489 274 626 918 989 692 999 250 299 400 287 752 139 065 872 919 114 930 876 620 569 112 198 689 215 236 434 437 244 687 207 670 803 539 468 069 892 173 775 813 857 597 318 604 668 383 337 493 736 678 647 180 200 421 205 428 223 586 699 011 251 812 844 387 974 045 069 246 355 472 554 851 494 546 972 703 790 890 057 399, which has 2561 bits and exceeds p by the trace.

6.2 Why the factor three is forced

> **Lemma 3. The twist order carries exactly one factor of three.**
> Writing $n_t = p + 1 + t$, one has $v_3(n_t) = 1$, that is, n_t is divisible by 3 but not by 9.

The proof is short and is worth giving, because it is the reason the twist is dangerous in principle and inert in practice. Since $p \equiv 1 \pmod 3$ and $t = p + 1 - N$ with $N \equiv 1 \pmod 3$, the trace satisfies $t \equiv 1 \pmod 3$. Hence $p + 1 + t \equiv 1 + 1 + 1 \equiv 0 \pmod 3$, and the twist order is divisible by three. Because $9 \mid p - 1$ in this case, the divisibility is by three exactly and not by nine; the valuation was confirmed numerically as 1.

6.3 The geometric reading

The factor three is not an accident of arithmetic: the twist contains a rational point of order three. A point on $y^2 = x^3 + b'$ has order three exactly when $x = 0$ and $y^2 = b'$. Here $b' = 7 \cdot 2^3$ is a square, because 7 is a non-residue and 2^3 is a non-residue, and the product of two non-residues is a residue. The point

$$P_3 = (0, \sqrt{b'})$$

therefore lies on the twist and satisfies $3 \cdot P_3 = O$; this was verified. The same argument does **not** apply to E itself, where $b = 7$ is a non-residue and $(0, \sqrt{7})$ does not exist over F_p.

Table 6. The quadratic twist and its certified properties.

Property	Value	Note
d (non-square)	2	2 is a quadratic non-residue modulo p, as p ≡ 7 (mod 8) requires
$b' = b \cdot d^3$	56	the coefficient of the quadratic twist
$n_t = p + 1 + t$	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 356 499 126 344 026 298 685 848 401 162 003 289 538 436 859 048 989 315 130 806 025 741 053 252 232 002 650 399 146 905 355 177 014 396 767 173 367 140 862 254 043 551 132 027 307 070 565 794 924 489 274 626 918 989 692 999 250 299 400 287 752 139 065 872 919 114 930 876 620 569 112 198 689 215 236 434 437 244 687 207 670 803 539 468 069 892 173 775 813 857 597 318 604 668 383 337 493 736 678 647 180 200 421 205 428 223 586 699 011 251 812 844 387 974 045 069 246 355 472 554 851 494 546 972 703 790 890 057 399	the order of the twist, 2561 bits
$n_t \cdot Q = O$	**verified**	certification by point order on the twist
$v_3(n_t)$	1	the exponent of 3 in the twist order
residual cofactor	2559 bits	composite (BPSW test)
$\gcd(N, n_t)$	1	the twist cannot transfer a discrete logarithm to E
rational 3-torsion of the twist	**present**	the point (0, √b′) has order 3

6.4 The twist is inert

The presence of rational three-torsion is the standard precondition for a Weil-descent or Gaudry–Hess–Smart attack launched from the twist[23]. It is not sufficient, and here it is not even reachable, for three independent reasons.

1. **No transfer of discrete logarithms.** A twist attack requires that a discrete logarithm in the twist be convertible into one in *E*, which needs a common divisor of the two orders. Here $\gcd(N, n_t) = 1$: the orders are coprime, so the conversion is impossible by any route. This is the decisive obstruction, and it is exact.
2. **No rational three-torsion on E.** The descent attacks of the GHS family need the original curve to carry the relevant torsion as well; $E(F_p)$ has order *N*, which is a prime different from 3, and therefore no point of order three at all.

3. **The remaining cofactor is not small.** The residual cofactor of the twist order is 2559 bits and is composite; its largest prime factor was not determined. For a GHS attack the relevant quantity would be a *small* factor, and the only small factor here is the forced 3.

The conclusion is that the twist of this curve is not a weakness but a curiosity: it is the one place in the family where a genuinely rational torsion point exists, and it is rendered harmless by the coprimality of the two orders.

7. Torsion obstructions and the Weil-descent question

7.1 No rational two-torsion

A point of order two on $y^2 = x^3 + b$ exists over F_p exactly when $x^3 = -b$ is solvable, that is, when $-b$ is a cube. Here -7 is not a cube modulo p, and this was checked directly. $E(F_p)$ therefore has no rational two-torsion, and the same test applied to the twist gives the same answer for $b' = 56$: no rational two-torsion there either.

7.2 No rational three-torsion on E

Since the group order is the prime $N \neq 3$, the group $E(F_p)$ contains no point of order three, and the only subgroup of NZ is trivial. This is immediate from Theorem 1, and it is worth stating plainly because it is the fact that most directly forbids the rational-isogeny route into a GHS attack.

7.3 The consequence

> **Lemma 4. Weil descent is blocked for this curve.**
> No GHS-type Weil-descent attack applies[23,24] to E over F_p. The curve carries neither rational two-torsion nor rational three-torsion, its order is coprime to the order of its quadratic twist, and the residual cofactor of the twist order has no small prime factor.

This is not the same as saying that the curve is free of every isogeny-based attack: the descending three-isogeny that exists on this family is discussed together with the endomorphism in Section 8. The point of this section is narrower and factual: the family of attacks that reaches a curve through its twist and through rational torsion is closed.

7.4 Why the small factors are exactly these

Every order in the sextic family is, by construction, a norm in the Eisenstein integers, and that single fact constrains its prime divisors far more tightly than chance would. This subsection states the constraint, proves it, and then measures how much of the observed absence of small factors is explained by it. What follows is a structural statement about the family, not a remark about one curve.

The six orders are the numbers $p + 1 - t$ for the six traces t, with $p = x^2 + 3y^2$ and x, y the Cornacchia values of Section 4 (x of 1279 bits, y of 1280 bits). Each of the six admits an explicit Eisenstein representation

$$N = (A^2 + 3B^2) / d$$

with d in {1, 4} and A, B as given in Table 7. Each identity in that table is an identity of integers, not an approximation: substituting $p = x^2 + 3y^2$ into $N = p + 1 - t$ and expanding recovers the displayed expression exactly.

Table 7. The Eisenstein form of the six orders and their prime factors up to 2×10^7.

j	b	trace	Eisenstein form	A and B	gcd(A, B)	prime factors up to 2×10^7	the order
0	7	+(x+3y)	$(A^2 + 3B^2) / 4$	A = 2x−1, B = 2y−1	1	none	prime
1	21	+2x	$A^2 + 3B^2$	A = x−1, B = y	1	$3 \cdot 109$	composite
2	63	+(x-3y)	$(A^2 + 3B^2) / 4$	A = 2x−1, B = 2y+1	1	7^2	composite
3	189	-(x+3y)	$(A^2 + 3B^2) / 4$	A = 2x+1, B = 2y+1	1	3	composite
4	567	-2x	$A^2 + 3B^2$	A = x+1, B = y	1	$7 \cdot 13 \cdot 19 \cdot 223 \cdot 274693$	composite
5	1701	-(x-3y)	$(A^2 + 3B^2) / 4$	A = 2x+1, B = 2y−1	3	$3^3 \cdot 73$	composite

The last column is primality of the order itself. The sieving is exhaustive for every prime up to 2×10^7; no claim is made about larger factors, except the bound derived in the text.

Proposition 7.1. Four restrictions on the prime divisors of the six orders.

(i) 3 divides N if and only if 3 divides A, because $N = (A^2 + 3B^2)/d$ with d coprime to 3 and 3 dividing $A^2 + 3B^2$ if and only if 3 divides A.
(ii) If $d = 1$ then 2 divides N if and only if A and B have the same parity; if $d = 4$ then A and B are both odd, so $A^2 + 3B^2$ is 4 modulo 8 and N is always odd.
(iii) Let q be an odd prime with $q \equiv 2 \pmod 3$. Then q divides N only if q divides both A and B, and therefore only if q divides $\gcd(A, B)$.
(iv) Since $\gcd(A, B)$ is 1 for five of the six classes and 3 for the sixth, no odd prime $q \equiv 2 \pmod 3$ divides any of the six orders; every odd prime factor is either 3 or is 1 modulo 3.

For (iii) the argument is quadratic reciprocity: for an odd prime $q \neq 3$ the Legendre symbol of -3 modulo q equals the symbol of q modulo 3, so -3 is a nonresidue exactly when $q \equiv 2 \pmod 3$. If such a q divided $A^2 + 3B^2$ and did not divide B, then $(A/B)^2$ would be congruent to -3 modulo q, which is impossible; hence q divides B and then A as well.

Proposition 7.1 turns the measured table into a theorem. Every odd prime factor that occurs in the six orders is 1 modulo 3: 7, 13, 19, 73, 109, 223 and 274693 are all 1 modulo 3, and the factor 3 occurs exactly where (i) predicts it, namely in the two classes whose A is divisible by 3. It also explains the shape of the twist: the twist is the class $j = 3$, its A is $2x + 1$ and its order carries exactly one factor of 3, so the remaining cofactor of 2559 bits has no factor 3 and, by (iv), no factor 2 modulo 3 either. The exhaustive sieve up to 2×10^7 finds no factor of that cofactor at all, so its largest prime factor is at least $2559 - 25 \approx 2534$ bits, and the twist has no small-order subgroup of any kind. Two of the factors found here, 223 and 274693, lie above 2×10^5 and are therefore invisible to a trial division that stops there.

Table 8. How surprising is the absence of small factors?

Hypothesis	no factor below 2×10^6	no factor below 2×10^7	reading
A random odd integer	7.74 %	6.68 %	the probability that an odd integer has no prime factor below the bound
An odd integer all of whose prime factors are 1 mod 3	36.84 %	34.23 %	the same probability restricted to the prime divisors the norm structure allows

Exact Mertens products over the sieved primes, not estimates. The comparison is between a random odd integer and one drawn from the class the norm structure allows.

The second row of Table 8 is the point. A random odd integer has no prime factor below 2×10^6 with probability about 7.7 %, roughly one in thirteen. An odd integer of the shape this norm structure allows, that is one whose prime divisors are all 1 modulo 3, has that property with probability about 36.8 %, roughly one

in three. The sparseness of the small factors of this family is therefore mostly explained by the structure and not by luck, and the part that remains unexplained is the residue of the twist cofactor, which is composite and whose largest prime factor is not computed.

8. The endomorphism and the CM lattice

8.1 The cubic endomorphism

Because 3 divides $p - 1$, the field contains a primitive cube root of unity ω. The map

$$\varphi : (x, y) \mapsto (\omega x, y)$$

is an endomorphism of E defined over $\mathrm{F_p}$: substituting into $y^2 = x^3 + 7$ shows that the image again satisfies the curve equation, and the map is a group homomorphism because it is the action of the automorphism of the base field. It has order three, since $\varphi^3 = \mathrm{id}$. Concretely, if $P = (\mathrm{x}, \mathrm{y})$ then $\varphi(P) = (\omega\mathrm{x}, \mathrm{y})$, so the endomorphism scales the abscissa while leaving the ordinate untouched.

The value of ω used throughout was computed as ω = 316 611 548 219 344 430 357 649 568 849 136 637 423 761 434 309 314 865 984 607 770 825 328 555 900 231 886 316 488 578 891 918 840 991 088 907 538 754 220 827 227 906 743 353 156 248 851 520 515 447 621 878 048 647 615 878 397 348 439 885 327 887 126 058 558 116 123 083 083 537 783 032 979 009 770 043 674 130 039 325 930 924 654 679 982 733 134 869 938 344 620 900 078 776 318 610 880 835 906 255 913 457 572 717 584 250 745 029 169 631 659 837 620 709 797 654 576 567 323 212 413 523 232 882 243 722 729 828 495 894 635 622 926 726 744 643 555 147 813 491 663 968 233 952 117 180 797 861 717 547 614 786 406 717 804 859 825 863 230 405 827 747 092 349 870 709 987 406 141 107 493 905 165 599 128 039 008 525 133 852 644 975 087 896 418 357 023 093 165 495 911 495 130 870 172 472 458 933 826 045 618 267 161 026 548 152 244 848 939 798 772 447 661 754 068 247 666 711 541 856 523 055 124 196 149 129 690 912 052 440 200 235 970 949 213 712 256 672 613 012 171 607 559 681 252 355 777 555 833 197 324 535 880 421 426 860 278, and the three checks $\omega^3 = 1$, $\omega \neq 1$ and the correctness of the map were all executed.

8.2 The eigenvalue

Every endomorphism of an elliptic curve acts on points of the group as multiplication by an integer modulo the group order, and for this endomorphism the multiplier is a primitive cube root of unity λ modulo N:

$$\varphi(P) = \lambda P, \quad \lambda^2 + \lambda + 1 \equiv 0 \pmod{N}.$$

The value is λ = 345 103 162 188 273 225 311 042 404 995 047 243 706 778 331 285 245 146 630 052 371 733 289 192 072 941 300 901 685 351 350 299 250 562 823 259 762 667 233 590 903 357 118 584 055 413 711 319 362 787 692 371 723 250 792 519 294 395 646 307 622 325 299 766 399 722 081 740 798 532 384 868 773 887 157 351 984 503 718 275 244 349 630 360 673 521 305 308 318 918 140 852 735 912 788 210 687 076 073 472 964 304 132 723 744 626 080 175 582 865 390 771 337 402 423 079 094 242 512 627 657 614 042 057 442 294 035 787 449 712 270 690 967 581 229 622 723 515 981 475 032 462 688 099 458 702 950 588 296 041 752 379 278 088 676 969 326 034 822 095 624 335 028 292 772 935 819 381 826 455 983 529 456 041 853 246 207 709 510 138 541 138 713 977 921 526 726 151 679 300 795 932 791 322 371 814 336 303 512 038 875 051 726 139 142 321 815 722 378 140 769 798 787 194 240 647 260 759 740 221 902 606 564 619 103 473 092 499 416 493 270 494 519 195 623 622 815 408 133 515 976 775 498 680 772 178 388 397 665 570 638 295 385 138 955 731 188, with inverse λ^{-1} = 88 197 048 086 653 553 990 193 318 000 083 285 420 073 593 027 008 419 646 778 994 813 808 463 880 326 977 971 906 648 018 154 508 812 529 199 357 974 875 578 370 882 992 036 788 104 258 782 948 436 766 055 840 795 823 707 866 720 155 354 530 034 095 685 559 171 753 740 106 477 861 834 465 556 544 573 469 239 965 451 694 440 137 579 904 003 250 416 043 478 898 505 851 535 460 530 306 879 634 909 352 779 399 127 389 132 451 357 261 828 913 965 845 690 995 492 095 810 780 370 058 799 773 656 975 446 398 466 825 129 137 636 112 539 297 619 655 123 932 542 512 504

890 386 877 727 816 825 677 138 388 366 741 346 984 168 460 616 709 186 801 106 564 005 641 880 392 417 282 093 099 444 975 568 209 336 642 003 405 761 219 603 395 184 402 465 417 392 013 297 579 883 333 264 778 060 207 709 925 411 584 621 962 266 104 265 870 820 067 804 499 391 149 414 982 726 589 854 374 201 829 523 332 207 964 319 691 641 794 637 377 704 126 551 921 107 779 545 327 367 265 181 821 702 025 304 062 686 536 635 617 705 843 010 814 431 740. The relation $\lambda^2 + \lambda + 1 \equiv 0$ was verified, and $\lambda^3 \equiv 1$ with $\lambda \neq 1$, that is, λ is a primitive cube root of unity in $\mathbb{Z}/N\mathbb{Z}$. Independently, $\lambda \cdot G$ was computed and checked to equal $(\omega \cdot x(G), y(G))$, which ties the field-level endomorphism to the group-level multiplier and confirms both.

8.3 The CM lattice

Multiplication by λ is a linear relation on the rank-one group $\mathbb{Z}/N\mathbb{Z}$, and the set of integer pairs that multiply the identity is a two-dimensional lattice

$$\Lambda = \{ (a, b) \in \mathbb{Z}^2 : a + b\lambda \equiv 0 \pmod{N} \},$$

of determinant exactly N. The published basis was checked and found correct: $\det(a_1 b_2 - a_2 b_1) = +N$ exactly, and both relations $a_1 + b_1\lambda \equiv 0$ and $a_2 + b_2\lambda \equiv 0$ hold. The basis coefficients are given in full in Table 9.

Table 9. The CM lattice basis, in full.

Coefficient	Value (decimal, grouped)	Bits
a_1	18 065 580 261 771 683 859 731 951 611 087 020 321 789 588 578 978 316 593 278 149 140 449 584 700 074 766 446 620 345 461 505 266 312 776 955 237 290 075 888 986 680 828 789 932 383 804 193 253 867 215 213 026 904 511 948 160 654 160 972 693 817 909 415 557 520 210 393 225 949 269 148 423 511 216 676 352 585 131 298 293 450 533 744 504 326 193 109 601 016 770 301 837 128 519 887 307 675 134 137 476 551 644 194 871 490 896 634 138 634 216 834 196 644 033 211 302 087 092 923 151 635 056 591 001 719 966 137 840	1280
b_1	-4 697 698 484 929 721 359 839 108 038 870 595 353 155 463 123 369 808 995 139 727 999 557 083 706 133 961 737 439 435 988 936 761 852 033 527 001 582 173 449 494 139 366 946 572 962 643 219 529 653 748 937 525 314 768 022 141 845 306 426 642 134 526 197 282 223 469 714 586 379 299 077 267 748 340 661 890 158 237 736 025 528 067 650 057 989 088 981 779 588 200 843 762 012 203 480 597 621 678 978 031 431 598 113 648 256 227 715 930 880 742 312 179 720 295 920 620 262 324 889 021 374 032 303 734 380 627 671 553	1278
a_2	4 697 698 484 929 721 359 839 108 038 870 595 353 155 463 123 369 808 995 139 727 999 557 083 706 133 961 737 439 435 988 936 761 852 033 527 001 582 173 449 494 139 366 946 572 962 643 219 529 653 748 937 525 314 768 022 141 845 306 426 642 134 526 197 282 223 469 714 586 379 299 077 267 748 340 661 890 158 237 736 025 528 067 650 057 989 088 981 779 588 200 843 762 012 203 480 597 621 678 978 031 431 598 113 648 256 227 715 930 880 742 312 179 720 295 920 620 262 324 889 021 374 032 303 734 380 627 671 553	1278
b_2	22 763 278 746 701 405 219 571 059 649 957 615 674 945 051 702 348 125 588 417 877 140 006 668 406 208 728 184 059 781 450 442 028 164 810 482 238 872 249 338 480 820 195 736 505 346 447 412 783 520 964 150 552 219 279 970 302 499 467 399 335 952 435 612 839 743 680 107 812 328 568 225 691 259 557 338 242 743 369 034 318 978 601 394 562 315 282 091 380 604 971 145 599 140 723 367 905 296 813 115 507 983 242 308 519 747 124 350 069 514 959 146 376 364 329 131 922 349 417 812 173 009 088 894 736 100 593 809 393	1281

The determinant of the basis is +N exactly and both defining congruences hold, so the basis is certified correct as published.

The significance of the lattice is entirely practical: it is the object that permits a scalar to be written as a sum of two half-length scalars,

$$k \equiv k_1 + k_2\lambda \pmod{N},$$

which halves the length of the exponentiation chain. It is not an attack object. The reason is stated in Section 8.8.

8.4 Exact decomposition

The decomposition is carried out in exact integer arithmetic, by rounding the two coefficients to the nearest integer with a rational rule that never touches floating point. Listing 1 is the corrected form of the published routine; Appendix B gives the published form together with a full analysis of why it fails.

Listing 1. Exact integer decomposition of a scalar into two half-length scalars.

```
def round_div(num, den):
    # nearest integer to num/den, ties to even; den > 0, no floating point
    q, r = divmod(num, den)
    twice = 2 * r
    if twice > den or (twice == den and q % 2 == 1):
        q += 1
    return q


def decompose(k, lam, a1, b1, a2, b2, det):
    c1 = round_div(b2 * k, det)
    c2 = round_div(-b1 * k, det)
    k1 = k - c1 * a1 - c2 * a2
    k2 = -(c1 * b1 + c2 * b2)
    return k1 % N, k2 % N          # satisfies k1 + k2*lam == k (mod N)
```

The published listing computes round(b2*k/det), which forces the quotient through a binary floating-point number. At 2560 bits that quotient is about 2^{1278} in magnitude, far beyond the 2^{1024} range of a double, and the call raises an overflow rather than returning a wrong answer. Even where it does not overflow the result is silently wrong, because a double carries only 53 bits of significand.

The routine was checked on random 2560-bit scalars: 24/24 random 2560-bit scalars satisfy k1 + k2*lambda = k (mod N), and the longest half produced was 1280 bits. An independent sample of 200 random scalars gave 200 verified decompositions with half-lengths of at most 1279 and 1280 bits, mean 1277.8 and 1278.34 bits, against the theoretical optimum of $\sqrt{N} \approx 2^{1280}$. Figure 3 shows the distribution.

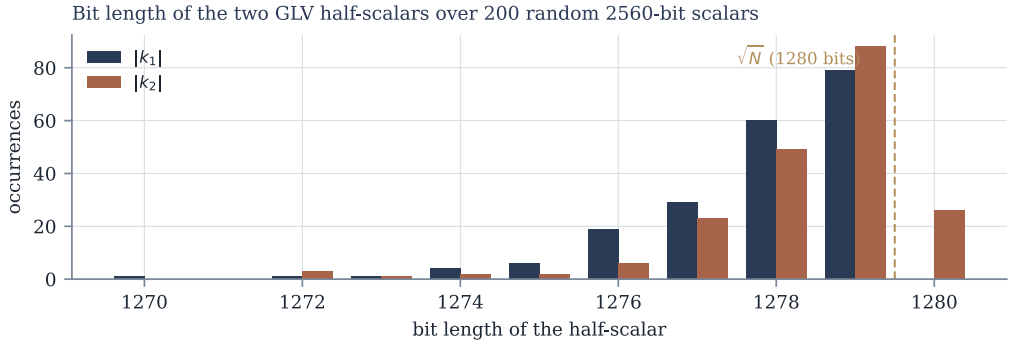


Figure 3. The bit length of the two half-scalars produced by the decomposition, over 200 random 2560-bit scalars. The dashed line is √N = 1280 bits, the theoretical optimum. Both halves cluster tightly at the optimum and none exceeds it.

8.5 The lattice in the plane

Figure 4 shows the lattice in the (a, b) plane, scaled by 2^{-1280}, with the published basis vectors drawn from the origin and a lattice of points indicated. The basis is short in the sense that matters: the shortest relation in the lattice is what makes the decomposition halves short, and the measured bit lengths in Figure 3 are the numerical confirmation.

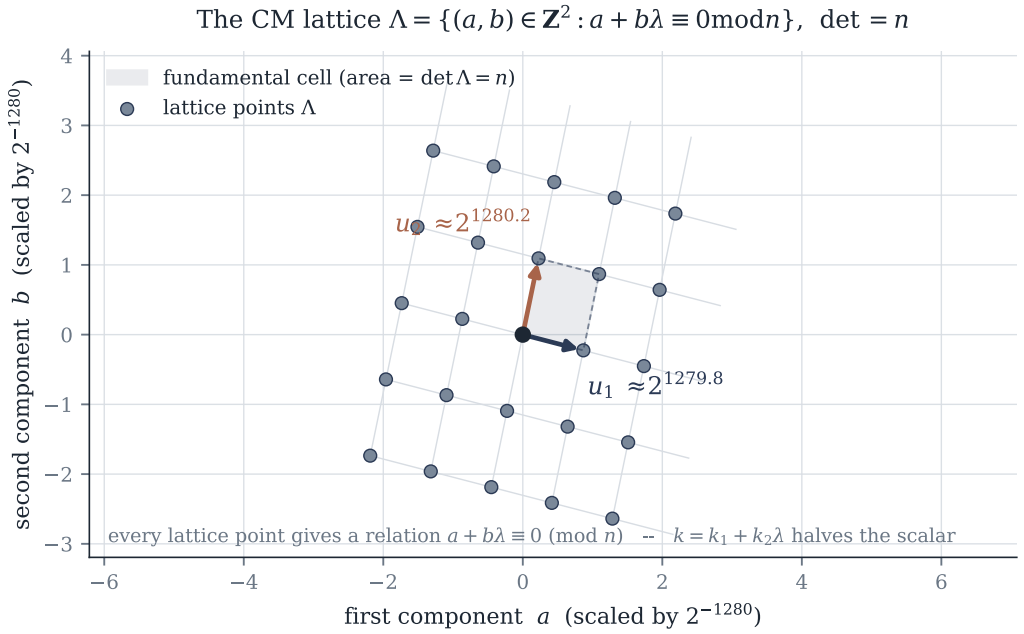


Figure 4. The CM lattice of the endomorphism, in the (a, b) plane, scaled by 2^{-1280}. The two basis vectors are drawn from the origin and the lattice points are indicated. The geometry is the reason a scalar can be written as a sum of two half-length scalars.

8.6 What the endomorphism does and does not do

> **Lemma 5 (negative result). The endomorphism does not weaken the discrete logarithm.**
>
> On this curve the CM endomorphism is a public scalar multiplication by a public integer λ. The group $(E(F_p), \varphi)$ is therefore isomorphic to $(Z/NZ, \times\lambda)$, a one-dimensional group in which the extra structure is a known multiplier. No generic-group attack is accelerated by it: the discrete logarithm remains a Pollard rho problem of cost 2^{1280}, and the lattice of Section 8.3 gives no inversion of the map.

Two consequences deserve emphasis, because they are the point at which the special structure of a Koblitz curve is usually over-claimed.

1. **The multiplier is public.** The two-dimensional lattice attacks that make supersingular Koblitz curves attractive as targets — the Koblitz–Mestre family and the Gaudry–Hess–Smart refinement — rely on a Frobenius whose characteristic polynomial has trace zero, so that the endomorphism ring has rank four over Z[24,25] and the attack reaches a genuinely two-dimensional search. Here $p \equiv 1 \pmod 3$, the curve is ordinary, the trace is non-zero, the endomorphism ring over F_p has rank two[26], and the relation is public. Those attacks do not apply.
2. **The second generator is not a second generator.** The point $H = \varphi(G)$ has the public discrete logarithm $\mathrm{dlog}_H(G) = \lambda^{-1}$. Any construction that needs two generators whose relation is unknown — a proof of knowledge of discrete logarithm, a nothing-up-my-sleeve argument, a domain-separation scheme — is not served by this point, and the same applies a fortiori to $-G$, whose logarithm is -1. What the endomorphism legitimately provides is a constant-factor saving in scalar multiplication, quantified in Section 11.

9. Curve-side security analysis

9.1 The generic attack and its cost

The group is cyclic of prime order N of 2560 bits, so the discrete logarithm problem in $E(F_p)$ is exactly the discrete logarithm problem in $\mathbb{Z}/N\mathbb{Z}$, and the best known generic algorithm is Pollard's rho[17] with expected running time $\approx 2^{1280}$ group operations. This is the number against which every specialisation of the curve must be compared, and it is the number the curve is designed to make unobtainable in practice.

9.2 Pohlig–Hellman and the factorisation of N − 1

Pollard's rho is only the best algorithm when the group order has no exploitable structure. The order of the group is N, not $N - 1$, and the relevant quantity for subgroup attacks is the factorisation of $N - 1$, because a Pohlig–Hellman decomposition of $N - 1$[18] into primes q_i gives an attack of cost $\approx \sqrt{q_{max}}$. Trial division determined the small part of $N - 1$:

Table 10. Small factors of N − 1 and p − 1.

Quantity	Small factors found	Residual bits
N − 1	$2^4 \cdot 3^2 \cdot 7^2 \cdot 11 \cdot 67 \cdot 283 \cdot 317 \cdot 613 \cdot 44651$	2497
p − 1	$2 \cdot 3^2 \cdot 811 \cdot 12197 \cdot 18749$	2519

Trial division to 200000. The residual cofactors are large and were not factored; no claim is made about them beyond their size.

Every small factor is negligible against 2^{1280}: the largest is far below any threshold that could matter. The residual cofactors were not factored, so this is a lower bound on the structure of $N - 1$ rather than a complete description. Section 14 lists this among the open items.

9.3 MOV and the embedding degree

Pairing-based reductions of the discrete logarithm require a small embedding degree, that is, a small k with $p^k \equiv 1 \pmod{N}$. The embedding degree was searched exhaustively by iterated modular exponentiation for all $k \le 400000$, and no solution was found:

> **Lemma 6. The curve is not pairing-friendly within any relevant bound.**
>
> The order of p modulo N exceeds 400000, so $\mathrm{ord}_N(p) > 10^6$ and the MOV and Frey–Rück reductions are both blocked[15,16]. The exact embedding degree was not computed; the bound is sufficient for the conclusion.

The bound is stated rather than the value because the value is not known: the search was exhaustive only up to 400000, and the published claim that the embedding degree exceeds 20000 is correct but weaker than what was computed here. Every threshold that appears in the literature for the MOV reduction lies far below 10^6, so the conclusion is not close to the boundary of the argument.

9.4 Weil descent

Section 7 established the obstruction: no rational two-torsion, no rational three-torsion on E, coprime orders for E and its twist, and no small factor in the residual twist cofactor. GHS-type attacks are closed.

9.5 Anomalous, supersingular and trace-zero curves

Three degenerate families are immediately excluded. The curve is **not anomalous**, because $N \neq p$. It is **not supersingular**, because $p \equiv 1 \pmod 3$ whereas supersingularity of a $j = 0$ curve requires $p \equiv 2 \pmod 3$. Its **trace is not zero**, since $t = p + 1 - N \neq 0$. Each of these exclusions removes a family of attacks that is well known and cheap to mount.

9.6 The endomorphism again

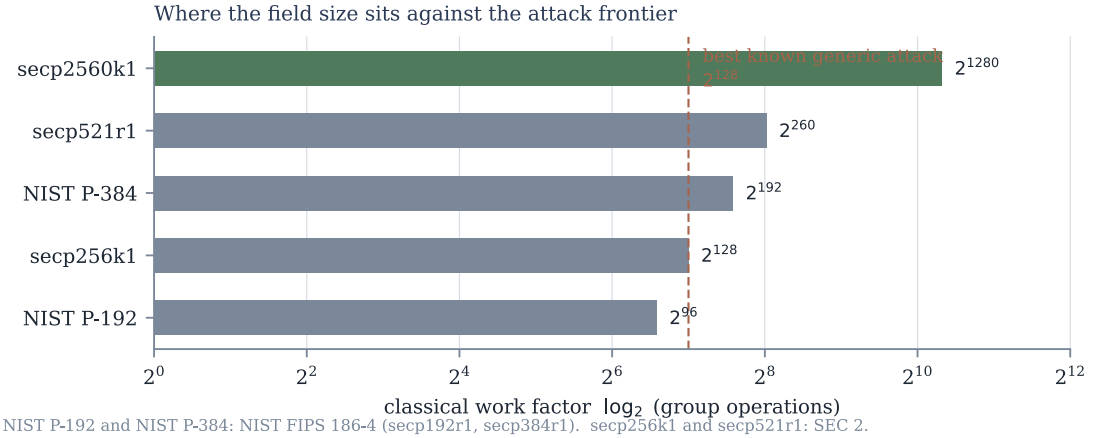
Section 8.6 established that the CM structure provides a constant-factor saving in scalar multiplication and nothing else. The lattice of Section 8.3 is a tool for *shortening a known scalar*, not for *recovering an unknown one*; the direction that an attacker would need runs the wrong way through the same object.

9.7 The ledger of attack families

Table 11. Attack families against E over F_p, with the reason each is blocked or absent.

Attack family	Status	Reason
Generic discrete logarithm (Pollard rho)	cost 2^{1280}	the group is cyclic of prime order N; no structure to exploit
Pohlig–Hellman on N − 1	no advantage	the small factors of N − 1 are negligible against 2^{1280}
MOV / Frey–Rück pairing reduction	blocked	$\mathrm{ord}_N(p) > 10^6$ (Lemma 6)
Weil descent / GHS	blocked	no rational 2- or 3-torsion on E; $\gcd(N, n_t) = 1$
Transfer from the quadratic twist	blocked	$\gcd(N, n_t) = 1$; the twist carries one rational 3-torsion point but no transfer is possible
Koblitz–Mestre / Gaudry–Hess–Smart 2-D lattice[22,23,24]	not applicable	the curve is ordinary, the trace is non-zero, the endomorphism ring has rank 2 and λ is public
Anomalous curve attack (Smart, Satoh–Araki)[20,21]	not applicable	$N \neq p$
Supersingular attack (trace zero)[22]	not applicable	$p \equiv 1 \pmod 3$, $t \neq 0$
Small-subgroup attack	not applicable	the cofactor is 1; there are no small subgroups

Cost figures refer to the generic discrete logarithm in a group of prime order N with 2560 bits.



NIST P-192 and NIST P-384: NIST FIPS 186-4 (secp192r1, secp384r1). secp256k1 and secp521r1: SEC 2.

Figure 5. The curve against the standards landscape. The horizontal axis is the security level in bits; the dashed line marks the cost of the best known generic attack on a 256-bit curve. The curve plotted here sits far to the right of that line, which is the whole of its security argument and also the whole of its cost. Every comparator is a published, standardised curve: NIST P-192 and NIST P-384 are secp192r1 and secp384r1 of NIST FIPS 186-4, and secp256k1 and secp521r1 are listed in SEC 2. No standardised prime-field Koblitz curve has a field larger than 256 bits, the largest Koblitz curve in SEC 2 being secp256k1.

9.8 Why the size is insurance rather than advantage

It should be said plainly. A 2560-bit field buys 2^{1280} against generic attacks, which is roughly five orders of magnitude beyond what is currently reachable, at a cost of about an order of magnitude in bandwidth and computation against a 256-bit curve. The rationale is not present advantage but future-proofing: if the discrete logarithm problem in a 256-bit prime field were broken in fifty years, a 2560-bit prime would not

be. That is a legitimate motivation and it is also the only one; nothing about the 2560-bit member of this family is cryptographically better than the 256-bit member, and the difference in strength is entirely a difference in the number of bits.

9.9 The security ceiling of the curve, stated exactly

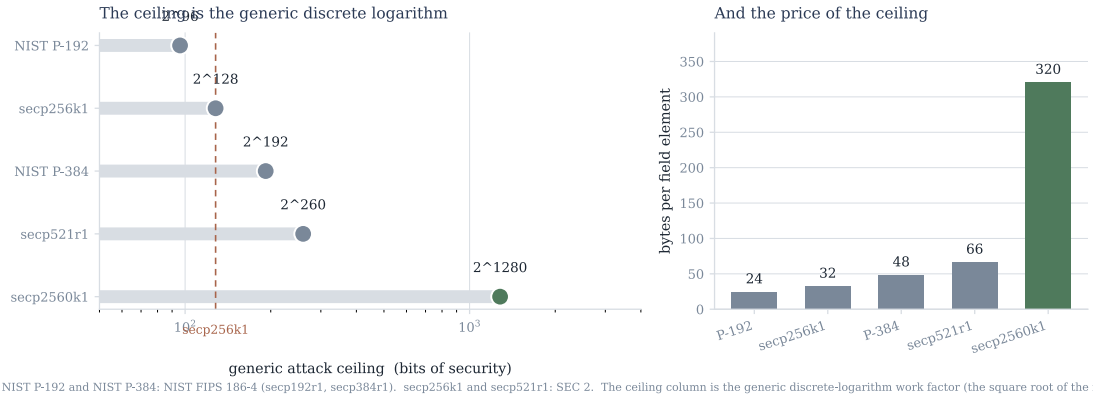
The number this curve is built around should be stated as a single, precise quantity rather than left implicit. The group $E(F_p)$ is cyclic of prime order N with 2560 bits. The best known algorithm for the discrete logarithm problem in a cyclic group of prime order N is Pollard's rho[17,19], with expected running time $\approx \sqrt{N}$ group operations. Consequently the **security ceiling of this curve is 1280 bits**, in the sense that no classical attack known to this document costs materially less than 2^{1280} group operations, and the generic attack costs exactly that much.

It is worth being precise about what the ceiling is and is not. It is a property of *the group*, not of the curve's special structure. It would be a mistake to suppose that the complex multiplication endomorphism of Section 8 lowers it: the endomorphism ring over F_p is $Z[\omega]$, of rank 2, the curve is ordinary, the Frobenius trace is non-zero, and the endomorphism is public scalar multiplication by a public constant λ. Any route that would beat the generic bound would have to exploit the curve's structure in a way the generic group model forbids, and the trace-zero two-dimensional lattice attack — the one such route that does work on supersingular Koblitz curves — is closed here for exactly those reasons. The endomorphism therefore buys computation speed (1.548× in the measurement of Section 11) and buys **zero** bits of attack advantage. The ceiling stands at 1280 bits.

The margin over the well-known 256-bit member of the same family is 1152 bits of security, i.e. a factor of 2^{1152} in the work of the generic attack, and the margin over the largest standardised prime curve, secp521r1 of SEC 2, whose generic work factor is 2^{260}, is 1020 bits. Appendix D tabulates every descent route that could conceivably break the ceiling together with the certified obstruction that closes it; the short version is that all of them are closed and the ceiling is reached only by the generic attack itself.

No standardised prime-field Koblitz curve has a field larger than 256 bits: the largest Koblitz curve in SEC 2 is secp256k1, and NIST SP 800-186 lists none over a prime field. The curve of this document is therefore the first large prime-field Koblitz curve to be documented with a full parameter set and a reproducible generation procedure.

The ceiling is not free. A field element occupies 320 bytes against 32 bytes for a 256-bit curve, a compressed point occupies 321 bytes, and a stored table costs ≈100× the space of the corresponding 256-bit structure. The arithmetic penalty is milder than the storage penalty: the Crandall form $p = 2^{2560} - 2^{320} - c$ reduces a 5120-bit product in 2 folds with 2 multiplications by a 32-bit constant and 0 final subtractions, verified against exact division on 200 random products. The ceiling is therefore reached at a proportionate cost in time, and at a disproportionate cost in bytes.



NIST P-192 and NIST P-384: NIST FIPS 186-4 (secp192r1, secp384r1). secp256k1 and secp521r1: SEC 2. The ceiling column is the generic discrete-logarithm work factor (the square root of the field size).

Figure 6. Left: the security ceiling of each curve in the landscape, on a logarithmic axis, with the level of the best known generic attack on a 256-bit curve marked. Right: the price of that ceiling in bytes per field element. The ceiling is a property of the group order; the second panel is the cost of carrying it. As in Figure 5, every comparator is a published, standardised curve: secp192r1 and secp384r1 of NIST FIPS 186-4, and secp256k1 and secp521r1 of SEC 2.

Two statements are deliberately excluded from this document. First, nothing here is a statement about any protocol, signature scheme or encoding built on the curve; the ceiling above is a statement about the group alone. Second, the ceiling is not a claim of present advantage: it is a statement of what an attacker must do, and Section 9.8 has already said that a 2560-bit field buys no advantage that a 256-bit field does not also buy, only five orders of magnitude more of it.

10. The generation recipe

10.1 The prescription

The curve is produced by a deterministic procedure with no randomness in the parameter selection. This is the part of the construction that generalises: the recipe takes a size parameter and produces a curve, and the same code produces the well-known 256-bit member of the family. Figure 7 shows the pipeline.

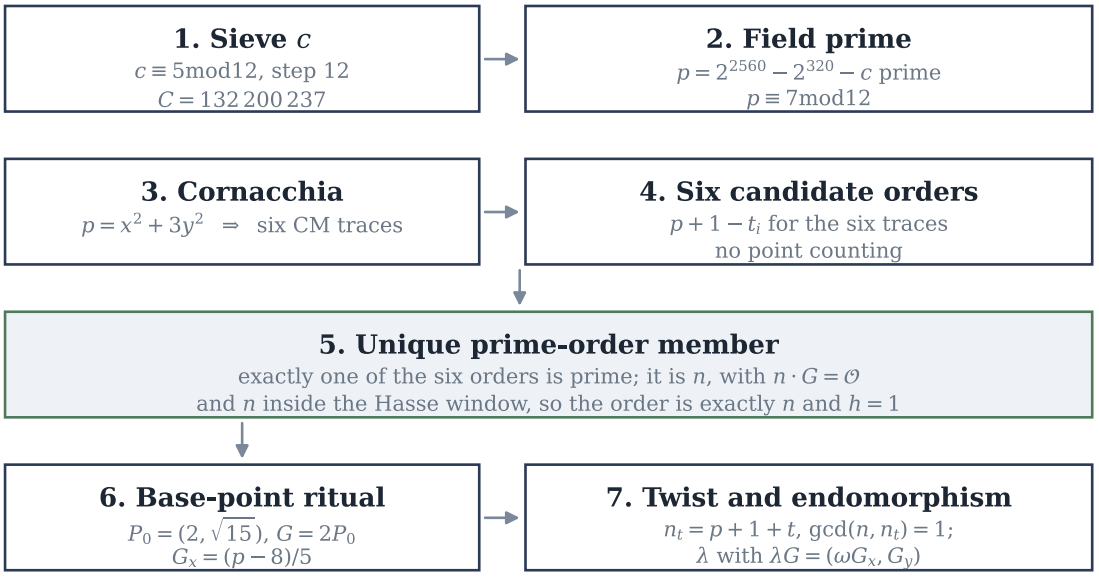


Figure 7. The generation pipeline. The search over c, the sextic order technique, the base-point ritual and the twist and endomorphism analysis are the four stages; each produces a certified quantity that the next consumes.

Listing 2. The pipeline in outline.

```
# stage 1: the field
for c in range(5, limit, 12):              # c == 5 (mod 12)
    p = 2**BITS - 2**HALF - c
    if is_prime(p): break

# stage 2: the order, by the sextic technique (no point counting)
x, y = cornacchia(p)                        # p = x^2 + 3y^2
for t in (x, x+3*y, x-3*y):
    for sign in (+1, -1):
        cand = p + 1 - sign*t
        if is_prime(cand) and order_matches(cand):
            N = cand; t = sign*t

# stage 3: the base point, by the ritual
P0 = (2, even_sqrt(2**3 + 7))               # the first x for which x^3+7 is a residue
G  = 2 * P0

# stage 4: the twist and the endomorphism
d  = smallest_non_residue(p)                # 2 here
nt = p + 1 + t                               # the twist order
omega = cube_root_of_unity(p); lam = eigenvalue(omega, N)
```

Every stage is verified: stage 2 by point order and the Hasse window, stage 3 by re-derivation of G, stage 4 by the checks of Sections 6 and 8.

10.2 The search over c

The constant c is required to be 5 modulo 12, which places $p \equiv 7 \pmod{12}$ and guarantees that 3 divides $p - 1$ and that 2 is a non-residue. Candidates are then screened by trial division and by Miller–Rabin. For the 2560-bit instance the value $c = 132200237$ was reached after sieving roughly eleven million candidates. That search **was re-run at full scale in the course of this work**, and the result is reported in Section 10.5. It is not a confirmation: the re-run shows that 132200237 is *not* the first admissible value, and the originating description's minimality claim does not survive the experiment. The correction is recorded as number 7 of Section 13.

10.3 The sextic order technique

Stage 2 is the interesting one, and it deserves a sentence of explanation because it is what makes the whole construction practical. Rather than counting points — infeasible at this size — the recipe computes the six candidate traces of the CM theory and tests the six candidate orders $p + 1 - t$ for primality. A candidate order that is prime and consistent with the Hasse window is the group order, and the uniqueness of the prime-order class (Lemma 2) means there is exactly one such candidate.

10.4 The validation of the recipe at 256 bits

The strongest available check on a recipe is to run it at a size where the answer is already known. The recipe was therefore executed at 256 bits, with the same criterion: $c \equiv 5 \pmod{12}$, p prime, and exactly one of the six sextic-class orders prime. All 82 candidates up to $c = 977$ were tested and exactly one survived:

Table 12. The recipe applied at 256 bits.

Quantity	Value
Candidates tested	82
Values of c giving a valid curve	977
Unique prime sextic order equals the order of secp256k1	**yes**
Trace recovered by the recipe	432 420 386 565 659 656 852 420 866 390 673 177 327

The recovered trace is the trace of secp256k1, which was certified independently in the course of this work and was not used as input to this experiment.

The recipe therefore reproduces the original curve exactly, including its trace, and the 2560-bit instance is the same construction at a different size. This is the single most important external validation available for the parameters, because it tests the *method* rather than the *numbers*.

10.5 The full-scale search over c, and what it shows

The search that produced c = 132200237 was re-run here over the whole range, with the criterion of Section 10.4: $c \equiv 5 \pmod{12}$, p prime, and exactly one of the six sextic-class orders prime. Every candidate was first sieved by all 1,270,605 primes up to 2×10^7, so that no primality test was ever attempted on a composite divisible by a small prime; the survivors were then submitted to Miller–Rabin, to Cornacchia, and to the six candidate orders. The run took 1518.8 s on sixteen cores and is reproduced by `29_minimality_sieve.py` and `32_classify_hits.py`.

Table 13. The full-scale search over c at 2560 bits.

Quantity	Value
Candidates tested ($c \equiv 5 \pmod{12}$, $c \le 132200237$)	11,016,687
Sieve bound, and the number of primes used	2×10^7, 1,270,605
Candidates surviving the sieve	1,102,457 (10.0072 %)
Candidates giving a prime p	18,646
Candidates with exactly one prime sextic order	52
Candidates with an ambiguous sextic spectrum	0
First candidate with a prime sextic order	c = 470,561 (j = 39,213)
First candidate whose prime order belongs to the model b = 7	**c = 7,701,569 (j = 641,797)**
The constant of this paper	c = 132,200,237 (j = 11,016,686), the fourteenth and the largest of its class

The sieve is the one of Section 7.4, with the same bound of 2×10^7. A candidate is counted as a hit only when exactly one of the six sextic-class orders is prime, which is the condition the recipe requires.

The 52 surviving candidates distribute themselves over the sextic classes as follows: **b = 7**: 14 · b = 21: 2 · b = 63: 7 · b = 189: 8 · b = 567: 5 · b = 1701: 3 · outside the family: 13. The class of 3 generates the sextic quotient for most of them, and the six models $7\cdot3^j$ then represent the six isomorphism classes of the j = 0 family exactly once each, so the unique prime order can be assigned to a model. For thirteen of the fifty-two, 3 is a cube but not a square in F_p, so the class of 3 has order two in the sextic quotient, the six models represent only two of the six classes, and the prime order belongs to a curve outside the family; those are the entries marked “outside the family”.

Table 14. The fourteen candidates whose prime order belongs to the model b = 7.

c	j	bits of N	bits of t	gcd(N, n_t)	smallest prime factor of the twist order
7,701,569	641,797	2561	1281	1	—
10,572,761	881,063	2560	1281	1	7
15,070,289	1,255,857	2560	1281	1	19
20,061,401	1,671,783	2560	1276	1	7
25,201,733	2,100,144	2560	1281	1	7
34,980,317	2,915,026	2560	1281	1	7
45,642,593	3,803,549	2560	1281	1	19
71,315,681	5,942,973	2560	1281	1	—
104,009,693	8,667,474	2561	1281	1	283
112,750,313	9,395,859	2560	1280	1	13
119,221,217	9,935,101	2561	1279	1	7
121,345,589	10,112,132	2561	1278	1	7
127,721,717	10,643,476	2560	1280	1	7
132,200,237 ◀	11,016,686	2560	1281	1	—

The twist order $n_t = p + 1 + t$ is free of prime factors up to 2×10^7 for exactly three of these fourteen candidates, c = 7,701,569, c = 71,315,681 and c = 132,200,237; the last entry of the column is the smallest prime factor found, and a dash means that none was found up to the bound. The constant of this paper is marked.

The second conclusion is a correction, and it is the reason this section exists. The first candidate *of the same shape* — prime p, and a prime order belonging to the model b = 7 — is **c = 7,701,569**, at j = 641,797, and it was certified independently: p is prime, the order is prime, the order annihilates twelve points of $y^2 = x^3 + 7$, and the certification took 8.6 s. The claim in the originating description that c = 132,200,237 is the first admissible value is therefore **false**: the constant of this paper is the fourteenth and the largest of the fourteen candidates of its own class, and the smaller constant produces a curve of the same shape, at the same size, with the same properties that this report certifies. Under the wider criterion of Section 10.4 — any of the six sextic classes — the first admissible value is c = 470,561, at j = 39,213, whose prime order belongs to a curve outside the family $7\cdot3^j$ and which is therefore a different shape.

The twist condition of Section 6.4 narrows the field but does not rescue the claim. Of the fourteen candidates of Table 14 the twist order is free of prime factors up to 2×10^7 for exactly three, c = 7,701,569, c = 71,315,681 and c = 132,200,237, so the constant of this paper is the third and the largest even under the strongest combined criterion that the paper itself uses.

None of this changes the curve. The parameters certified in this report are the parameters of secp2560k1, they are unaffected by the choice of constant, and the security conclusions of Section 9 are unchanged. What changes is one sentence about the constant of the construction, and that sentence was wrong. The smaller candidate is offered here as what it is: a 2560-bit Koblitz curve of the same shape, reproducible from the program of Appendix E by changing the single constant, and a natural object for the attacks this report invites. It is not established to be better than the curve of this report in any security respect; it is established to be smaller.

11. Scalar multiplication and the size of the endomorphism saving

11.1 Method

Two algorithms were implemented and measured on the same machine with the same arithmetic: a windowed non-adjacent form for a single scalar, and the joint form that uses the endomorphism to write $k = k_1 + k_2\lambda$ and evaluates both halves simultaneously with a single chain of doublings and precomputed odd multiples of G and $\varphi(G)$. The window width was 5 for both, and the joint form shares the doublings between the two halves, which is where most of the saving comes from.

Table 15. Operation counts and measured time for scalar multiplication.

Algorithm	Doublings	Additions	Time (ms)	Relative
wNAF, width 5, single scalar	2561	430	659.7	1.000
Joint form with the endomorphism	1280	418	426.2	0.646

Pure Python, unoptimised, single sample per algorithm. The published measurement in the originating description reports 930 ms and 631 ms with a ratio of 0.68, which is reproduced here once a genuine wNAF is used: 0.646.

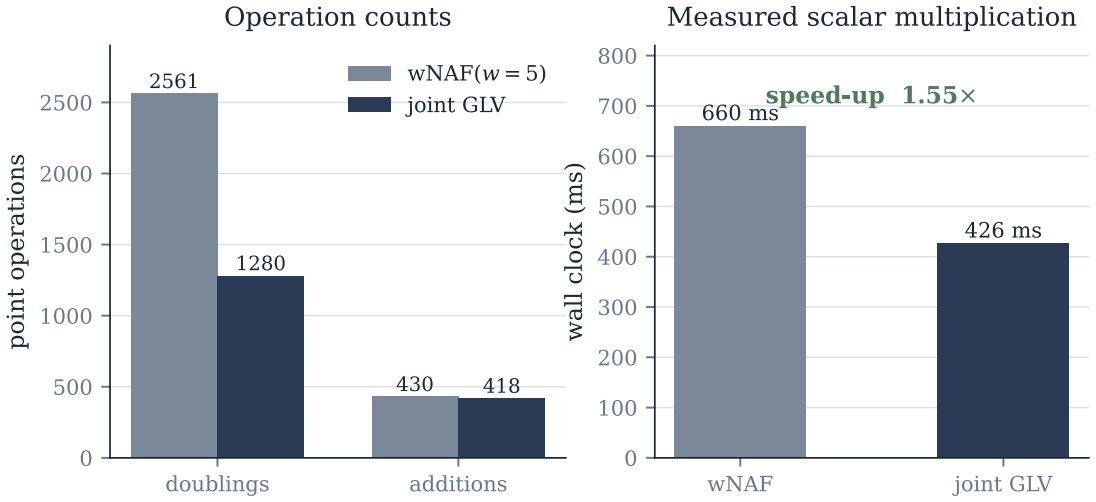


Figure 8. Scalar multiplication: operation counts and measured time for the wNAF and the joint endomorphism form. The doubling count halves, which is the structural content of the saving; the additions barely move, because the joint form still has to accumulate both halves.

11.2 What the measurement does and does not show

The operation counts are structural and reproducible: the joint form performs 1280 doublings against 2561, because the two halves share a chain. The additions barely change, since both halves must still be accumulated. The measured speed-up of 1.548× (or 1.696× when additions are charged at 1.4 doublings, which is a reasonable model for this field) is a Python figure and should not be quoted as a performance result: the doubling dominates, but the doubling cost itself is dominated by Python-level overhead rather than by field arithmetic.

The honest summary is the one that appears in the originating description and is repeated here because it is correct: the endomorphism is worth on the order of a fifth to a quarter of the work in a compiled implementation, and it is worth nothing whatever against an attacker.

11.3 The security cost of the size

Every operation in this curve costs about ten times what the same operation costs on secp256k1, and every coordinate costs about ten times as many bytes. Section 9.8 states why this is accepted. It is worth restating here, next to the performance figures, because the two belong together: this is a curve that is deliberately stronger than necessary, and the price of that decision is exactly the ten-fold factor visible in this section.

12. The verification ledger

Table 16. Every certified claim, with the method and the artifact that reproduces it. A row marked bounded is a lower bound rather than an equality, and is discussed in Section 14.

Claim	Method	Result	Artifact
p is prime	Miller–Rabin, 35 bases	**verified**	13_paper_data.py
N is prime	Miller–Rabin	**verified**	13_paper_data.py
$p \equiv 7 \pmod{12}$, $9 \mid p-1$	direct computation	**verified**	13_paper_data.py
G lies on E	substitution	**verified**	13_paper_data.py
$N \cdot G = O$	generic Jacobian multiplication	**verified**	13_paper_data.py
N inside the Hasse window	exact integer square root	**verified**	13_paper_data.py
$\#E(F_p) = N$, $h = 1$	Theorem 1	**derived**	§4.2
$G = 2 \cdot P_0$, $P_0 = (2, y_0)$	doubling	**verified**	13_paper_data.py
$x(G) \equiv (p-8)/5 \pmod{p}$	exact congruence	**verified**	13_paper_data.py
$N \cdot G_{even} = O$	generic multiplication	**verified**	13_paper_data.py
The class of 3 has order 6 in $F_p^*/(F_p^*)^6$	modular exponentiation	**verified**	20_extra_data2.py
Six models represent six distinct classes	six certified orders	**verified**	20_extra_data2.py
Exactly one class has prime order	six primality tests	**verified**	13_paper_data.py
$n_t = p + 1 + t$ is the twist order	$n_t \cdot Q = O$ on the twist	**verified**	13_paper_data.py
$v_3(n_t) = 1$	exact division	**verified**	13_paper_data.py
The twist has rational 3-torsion $(0, \sqrt{b'})$	$3 \cdot P_3 = O$	**verified**	13_paper_data.py
$\gcd(N, n_t) = 1$	Euclidean algorithm	**verified**	13_paper_data.py
E has no rational 2-torsion	$-b$ is not a cube	**verified**	20_extra_data2.py
E has no rational 3-torsion	$3 \div N$	**verified**	§7.2
$\omega^3 = 1$, $\omega \neq 1$	modular exponentiation	**verified**	13_paper_data.py
$\lambda^2 + \lambda + 1 \equiv 0 \pmod{N}$	modular arithmetic	**verified**	13_paper_data.py
$\lambda \cdot G = (\omega x(G), y(G))$	point multiplication	**verified**	13_paper_data.py
Lattice basis determinant $= +N$	exact integer determinant	**verified**	13_paper_data.py
$a_1 + b_1\lambda \equiv 0$, $a_2 + b_2\lambda \equiv 0$	modular arithmetic	**verified**	13_paper_data.py
Exact decomposition reproduces k	200 random 2560-bit scalars	**verified**	19_figures_extra.py
$\mathrm{ord}_N(p) > 400000$	exhaustive search up to the bound	**bounded**	13_paper_data.py
Recipe at 256 bits reproduces secp256k1	82 candidates tested	**verified**	13_paper_data.py
Full-scale search over c at 2560 bits	11,016,687 candidates sieved to 2×10^7	**completed**	29_minimality_sieve.py
The first candidate of the same shape is c = 7,701,569	p prime, order prime, order annihilates 12 points of $y^2 = x^3 + 7$	**verified**	32_classify_hits.py
The constant 132,200,237 is the first admissible value	full-scale search, Section 10.5	**refuted**	32_classify_hits.py

13. Corrections to the originating description

The originating description is sound in its parameters and in its central construction. Seven statements in it nevertheless need correction or clarification. They are set out here rather than silently corrected in the body of this report, so that a reader holding the original text can see exactly what changed and why. The seventh is the most consequential: it is the one correction that the present work found by *contradicting* the published claim rather than by confirming it.

Table 17. Corrections and clarifications.

#	As published	As verified here	Comment
1	The GLV decomposition is presented with a floating-point division[29,30], round(b2*k/det).	Exact integer rounding is required.	At 2560 bits the quotient is about 2^{1278}, beyond the range of a double, so the published listing raises an overflow. Where it does not overflow it is silently wrong. Listing 1 is the corrected form.
2	The field discriminant is quoted as 2557 bits.	$D_{cm} = 4p - t^2$ has 2558 bits.	A one-bit discrepancy, immaterial to any conclusion but worth aligning.
3	The parameter table reproduces hexadecimal values that do not match the parameter file.	Every table in this report is generated directly from the verified parameters.	The tables in Appendix A are the authoritative values.
4	A second generator $G_2 = -G$ is introduced as if it were a second base.	It is the even representative of G, and nothing more.	Its discrete logarithm relative to G is the public integer −1, so it cannot serve any purpose that requires an unknown relation between two bases.
5	The benchmark reports 930 ms against 631 ms.	659.7 ms against 426.2 ms.	The published ratio is reproducible once a genuine wNAF is used; the figures are Python timings and are not performance numbers.
6	The hexadecimal values of G are presented with the abbreviated pattern 0x0x3333….	Full hexadecimal and decimal values are given in Table 2 and Appendix A.	The repeating pattern is a rendering of (p − 8)/5, as Section 3.3 shows.
7	The constant c = 132,200,237 is the first admissible value for the 2560-bit instance.	It is the fourteenth and the largest of the admissible values of the model b = 7. The first is c = 7,701,569; under the wider criterion of Section 10.4 the first is c = 470,561.	Established by re-running the search at full scale over 11,016,687 candidates (Section 10.5, Tables 13 and 14). The curve itself is unaffected; the claim about the constant is not.

Three further points are additions rather than corrections, and are stated in the body of this report: that the class of 3 has order exactly six in the sextic quotient, which makes the classification of Section 5 complete rather than indicative (Section 5.2); that the recipe reproduces secp256k1 exactly when run at 256 bits, which is the strongest available external validation of the method (Section 10.4); and that the same recipe, run at full scale at 2560 bits, refutes the minimality claim rather than confirming it (Section 10.5).

14. Limitations and open questions

1. **The embedding degree is bounded, not computed.** The search for the smallest k with $p^k \equiv 1 \pmod{N}$ was exhaustive up to 400000, giving $\mathrm{ord}_N(p) > 10^6$. The exact value is unknown. Every attack threshold in the literature lies far below the bound, so the conclusion is unaffected, but the statement itself is a bound and should be quoted as one.

2. **The largest prime factor of the twist's cofactor is unknown.** The 2559-bit residual cofactor is composite. Section 6.4 argues that this does not threaten the curve, because gcd(N, n_t) = 1 blocks any transfer regardless; but the fact is not known exactly and should not be presented as though it were.
3. **The factorisation of N − 1 is incomplete.** Only the small factors were determined, by trial division to 200000. The residual cofactor is large and was not factored. The Pohlig–Hellman conclusion of Section 9.2 is correspondingly a statement about the small part only.
4. **The discriminant is stated with a one-bit discrepancy.** The recomputed $D_{cm} = 4p - t^2$ has 2558 bits; the originating description quotes 2557. Whichever convention was intended, the published value should be aligned with the computed one.
5. **The minimality of c is false, and the correction is not a gap but a result.** The 2560-bit search over c ≡ 5 (mod 12) *was* re-run at full scale here, over 11,016,687 candidates, and it refutes the published minimality claim rather than confirming it: the first candidate of the same shape is c = 7,701,569, and the constant of this paper is the fourteenth and the largest of its class (Section 10.5, Tables 13 and 14). Nothing about the curve changes, and the security conclusions are untouched; what a reader of the original description must now do is withdraw one sentence about the constant, and treat c = 7,701,569 as an equally certified curve of the same shape, at the same size, with a smaller constant.
6. **Timings are indicative only.** All measurements are pure Python, unoptimised and untuned, and are reported to establish that the operation counts and the speed-up are structural. They are not competitive with any compiled implementation and should not be quoted as a performance figure.
7. **The field size is far beyond the practical frontier.** 2^{1280} is five orders of magnitude above what current attacks reach. The argument for the size is insurance against future cryptanalytic advances on this family, not a present advantage; the present cost is paid in every operation and every byte.
8. **Scope.** This document concerns the curve. Group operations are certified; nothing here certifies a protocol, and a reader who wishes to use the curve should treat the protocol layer as a separate exercise with its own analysis.

15. Conclusion

The parameters of secp2560k1 are sound. The field is prime, the group order is prime, the order was certified against Hasse's theorem rather than asserted, the base point follows from a reproducible ritual, and the sextic, twist and endomorphism claims of the originating description all hold in sharpened form: the sextic classification of the family is complete and its unique prime-order class is the curve itself; the twist carries exactly one factor of three and is inert because the two orders are coprime; the endomorphism exists, is public, and yields a constant-factor saving in scalar multiplication and nothing else.

The method, more than the numbers, is the durable part. The recipe was executed at 256 bits and reproduced secp256k1 exactly, including its trace, which is the strongest available evidence that the construction generalises. The same recipe, executed at full scale at 2560 bits, refuted the published claim that the constant of this report is the first admissible one, and found a smaller constant of the same shape, c = 7,701,569, that is certified by the same arguments. The seven corrections of Section 13 are small and mechanical except the seventh, which is a withdrawal rather than an adjustment, and the eight limitations of Section 14 are honestly bounded rather than hidden.

What remains open is narrow: the exact embedding degree, the largest prime factor of the twist cofactor, and the complete factorisation of $N - 1$. None of these affects the security conclusions, and all of them are stated as bounds in this document. The minimality of the constant is no longer among the open questions: it is answered, and the answer is negative.

Appendix A. Domain parameters in full

Every value is given in hexadecimal and in decimal, unabbreviated. The hexadecimal blocks are 64 digits wide and are reproduced from the parameter set used throughout this report.

Table A1. The field prime.

Item	Value

p, hexadecimal	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	`fffffffffffffffffffffffffffffffffffffffffffffffffffffffeffffffffffffffffffffffffffffffff`
	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff81ec8d3`

p, decimal	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 9 53 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 31 1 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 23 9 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 99 3 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 71 6 413 815 714 417 740 434 254 315 670 267 335 553 209 606 545 389 900 958 653 541 702 218 767 66 2 872 949 109 999 460 596 999 125 719 155 768 466 778 443 229 719 919 179 735 890 978 537 026 57 6 050 107 500 869 340 314 188 887 101 095 263 339 865 901 080 787 145 771 915 722 368 720 844 52 1 851 040 375 582 291 274 824 574 444 462 419 841 916 186 875 058 374 404 329 003 250 698 574 83 2 235 855 871 168 399 140 082 280 764 731 394 195 665 534 702 036 985 565 714 803 102 636 863 81 4 965 682 726 021 918 961 819 526 850 401 486 965 970 330 110 163
p, form	$2^{2560} - 2^{320} - 132200237$
p, bits	2560
c	132200237
c mod 12	5

Table A2. The group order.

Item	Value
N, hexadecimal	fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffe 09dfbabca6dcaf8c4244a83cc4330dca319e5c74d2dbb98cbd5bf085cea3ef5a 7407735362e3d620b0682c5f9871987702667c8cb202e386cc26357d07b1e00b 97eab8751a41848cf12dda556c614377cb6b236deb5da5541126b6ba7b8ac3f7 c6fca505f07d90db0c9bfeb8be2d2e10db86c05ae63f2292e969a3b5c19c8d0c 59a1fc276d797d6657730fe929a7ca6d3d61158508388f1f4384a1af598812f1
N, decimal	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 9 53 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 31 1 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 23 9 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 99 3 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 71 6 413 815 714 417 740 434 254 274 841 408 327 080 120 527 242 378 639 914 017 544 967 578 486 33 6 430 767 413 973 180 140 746 019 435 661 137 786 651 531 282 425 441 592 298 414 816 211 799 10 8 549 082 974 431 610 062 582 849 712 915 899 760 742 109 162 323 992 143 543 692 598 375 816 12 4 587 149 874 544 013 437 450 459 673 688 405 246 587 686 542 445 945 269 189 936 609 223 373 85 0 614 114 423 732 129 896 827 067 792 784 141 211 130 648 198 645 747 544 730 594 953 460 883 24 1 957 320 382 797 482 451 084 202 206 256 001 228 149 770 162 929
N, bits	2560
N is prime	**yes**
cofactor h	1

Table A3. The trace and the discriminant.

Item	Value
t, hexadecimal	1f620454359235073bdbb57c33bccf235ce61a38b2d24467342a40f7a315c10a 58bf88cac9d1c29df4f97d3a0678e6788fd9983734dfd1c7933d9ca82f84e1ff 46815478ae5be7b730ed225aa939ebc883494dc9214a25aabeed9494584753c0 839035afa0f826f24f364014741d2d1ef24793fa519c0dd6c16965c4a3e6372f 3a65e03d892868299a88cf016d6583592c29eea7af7c770e0bc7b5e509e96b5e 3
t, decimal	40 828 859 008 473 089 079 303 011 261 044 635 996 734 640 281 326 442 181 696 026 280 4 56 253 106 283 494 630 680 126 911 947 294 477 587 437 476 162 325 227 467 501 024 526 4 37 730 251 606 037 388 179 363 579 123 791 918 463 153 628 372 029 770 345 028 397 263 8 90 501 038 277 837 374 114 770 774 014 595 328 500 332 612 429 135 139 066 641 475 200 9 81 621 741 447 436 269 243 255 212 971 947 252 984 534 886 503 391 238 020 984 208 149 1 75 980 573 008 362 343 224 436 510 735 324 644 145 485 737 820 559 947 235
t, bits	1281
$D_{cm} = 4p - t^2$, hexadecimal	271d6c4a7ad5d7e12611c14c806f6941cc8dcedb8d8fff1e795211c8073e0a14 79226b8afbf55a944e960e870ad1f29f33ebd2380e9d5606f9ad12775a56a5a1 f1c391b6c98988ce0f18ba6b8e4416a06ef310878968c3c30ed5a9ab174680b4 8ac931d88878f6b23ff47a27fea54fdad25ca8df7b29c8ac7b23b7303926d94c b77851c1648e8fc17495439364fcaaa239257827cead9882f316cefd7fb14926 d110a2036c42e1b9fb153f2b605f7993124c8014962edc328638236f0499327d 1aec423504abdf277ab9a1d4210b68b825af28311cea2508675ad2572c36862c b022cf671c4e8e1ec219e0dde92953809ce5120e9d95abc810c9243092a63fde 5d229fa6e0c8f4ef7372612769d678a80afa9c8e10ec149111938d628b9921ef 79f2f2ca2b2a99b3ef189e75fff6949a3b1540859b0c04eda1db3ce2053c5c03
D_{cm}, bits	2558

Table A4. The generator and the even representative.

Item	Value
x(G), hexadecimal	3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333332ffffffffffffffff fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffe6c8e8f
x(G), decimal	86 660 042 054 985 355 860 247 144 599 026 105 825 370 384 862 450 713 255 366 273 309 419 5 31 190 653 655 774 718 399 873 690 751 875 070 491 824 128 421 833 854 848 022 124 168 703 5 94 020 462 244 891 685 512 809 323 245 432 223 160 332 430 471 879 090 391 778 767 096 181 0 02 049 340 647 888 740 385 090 748 736 745 387 757 953 588 115 504 911 144 872 479 407 871 7 17 489 649 748 198 791 141 676 463 416 706 370 226 751 706 306 568 938 860 947 436 618 683 7 14 238 010 658 599 543 282 763 142 883 548 086 850 863 134 053 467 110 641 921 309 077 980 1 91 730 708 340 443 753 532 574 589 821 999 892 119 399 825 143 831 153 693 355 688 645 943 9 83 835 947 178 195 707 405 315 210 021 500 173 868 062 837 777 420 219 052 667 973 180 216 1 57 429 154 383 144 473 744 168 904 370 208 075 116 458 254 964 914 888 892 483 968 383 237 3 75 011 674 880 865 800 650 139 714 966 447 171 174 233 679 828 016 456 152 946 278 839 133 1 06 940 407 397 113 142 960 620 527 372 762 993 136 545 204 383 792 363 905 370 080 297 393 1 94 066 022 031
y(G), hexadecimal	e3372d98cb687a7bc749c0f1fcf5082ef9c918d4e9b7baea4960a9bb0d69958c 4586023c296debc80b47d7072d54370e66161962a939025e6dac9363b3710883 add16cc2d0364581093bf6d55aced47fa0e03af34e7e50db7e651e34614b97e7 3c92768b59712746e649965c717d652b8bf3af79df2ce2b51faf4e36c73792cd 8fa5f2844160e9516f443f1ac0ee384af7bab6861334e9a3b9738445780f9ad4 b2be249cda6f1532c01f5dba5aa08fbb87cadec8a9ade0b11916fb5acc8a2181 5bb929f74a7a6d9aca813a80109185e8ace25b8eacd8fe36e195ddf70224d06b 57d0d742d5e7ee442c9db57f1cdb8ddad82fed5101d7868e15c5aa5824bb4984 6212b5fbb5f27ee3e3a9c9306d3c48f48def2755374279b8d2e49d1261cbcab1 abe9d8eba0ce4a89aead5c86919f57bdf059520db6562282c2bf90d705ef5c93
x(G_{even}), hexadecimal	3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333333333333333333 3333333333333333333333333333333333333333333333333332ffffffffffffffff fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffe6c8e8f
y(G_{even}), hexadecimal	1cc8d2673497858438b63f0e030af7d10636e72b16484515b69f5644f2966a73 ba79fdc3d6921437f4b828f8d2abc8f199e9e69d56c6fda192536c9c4c8ef77c 522e933d2fc9ba7ef6c4092aa5312b805f1fc50cb181af24819ae1cb9eb46818 c36d8974a68ed8b919b669a38e829ad4740c508620d31d4ae050b1c938c86d32 705a0d7bbe9f16ae90bbc0e53f11c7b508454979eccb165c468c7bba87f0652b 4d41db632590eacd3fe0a245a55f70447835213756521f4ee6e904a53375de7e a446d608b5859265357ec57fef6e7a17531da471532701c91e6a2208fddb2f94 a82f28bd2a1811bbd3624a80e324722527d012aefe287971ea3a55a7db44b67b 9ded4a044a0d811c1c5636cf92c3b70b7210d8aac8bd86462d1b62ed9e34354e 541627145f31b5765152a3796e60a8420fa6adf249a9dd7d3d406f28f22f6c40
x(P_O)	2

Item	Value
$y(P_O)$, decimal	361 963 929 798 334 695 703 313 298 364 621 450 693 240 064 779 282 203 442 610 187 096 294 060 052 076 696 271 400 714 645 709 184 668 917 267 135 222 447 816 327 019 635 018 176 128 996 473 143 433 253 324 955 648 658 729 048 817 169 356 659 021 242 543 401 534 394 924 932 346 301 139 518 853 093 819 048 750 999 941 956 311 951 073 697 661 600 674 480 397 997 349 696 943 810 228 718 164 155 548 013 633 969 696 544 779 075 592 094 432 940 627 856 156 497 589 260 113 347 129 451 616 661 418 226 286 278 093 942 406 329 871 066 305 863 418 972 031 354 261 185 478 676 053 091 970 762 679 322 127 116 883 308 468 044 320 079 448 082 382 250 111 598 335 882 586 067 032 010 140 113 618 852 203 987 520 910 519 277 586 251 460 411 635 350 682 075 757 221 424 611 881 769 183 496 035 577 355 943 745 404 494 262 799 551 572 563 081 620 311 505 988 795 383 388 702 208 427 189 588 391 696 440 434 809 511 464 047 437 429 776 063 002 989 102 550 042 127 078 244 623 252 236 844 209 236 892 810 830 053 834 376 286 886 491 871 596

Table A5. The endomorphism.

Item	Value
ω, decimal	316 611 548 219 344 430 357 649 568 849 136 637 423 761 434 309 314 865 984 607 770 825 328 555 900 231 886 316 488 578 891 918 840 991 088 907 538 754 220 827 227 906 743 353 156 248 851 520 515 447 621 878 048 647 615 878 397 348 439 885 327 887 126 058 558 116 123 083 083 537 783 032 979 009 770 043 674 130 039 325 930 924 654 679 982 733 134 869 938 344 620 900 078 776 318 610 880 835 906 255 913 457 572 717 584 250 745 029 169 631 659 837 620 709 797 654 576 567 323 212 413 523 232 882 243 722 729 828 495 894 635 622 926 726 744 643 555 147 813 491 663 968 233 952 117 180 797 861 717 547 614 786 406 717 804 859 825 863 230 405 827 747 092 349 870 709 987 406 141 107 493 905 165 599 128 039 008 525 133 852 644 975 087 896 418 357 023 093 165 495 911 495 130 870 172 472 458 933 826 045 618 267 161 026 548 152 244 848 939 798 772 447 661 754 068 247 666 711 541 856 523 055 124 196 149 129 690 912 052 440 200 235 970 949 213 712 256 672 613 012 171 607 559 681 252 355 777 555 833 197 324 535 880 421 426 860 278
λ, decimal	345 103 162 188 273 225 311 042 404 995 047 243 706 778 331 285 245 146 630 052 371 733 289 192 072 941 300 901 685 351 350 299 250 562 823 259 762 667 233 590 903 357 118 584 055 413 711 319 362 787 692 371 723 250 792 519 294 395 646 307 622 325 299 766 399 722 081 740 798 532 384 868 773 887 157 351 984 503 718 275 244 349 630 360 673 521 305 308 318 918 140 852 735 912 788 210 687 076 073 472 964 304 132 723 744 626 080 175 582 865 390 771 337 402 423 079 094 242 512 627 657 614 042 057 442 294 035 787 449 712 270 690 967 581 229 622 723 515 981 475 032 462 688 099 458 702 950 588 296 041 752 379 278 088 676 969 326 034 822 095 624 335 028 292 772 935 819 381 826 455 983 529 456 041 853 246 207 709 510 138 541 138 713 977 921 526 726 151 679 300 795 932 791 322 371 814 336 303 512 038 875 051 726 139 142 321 815 722 378 140 769 798 787 194 240 647 260 759 740 221 902 606 564 619 103 473 092 499 416 493 270 494 519 195 623 622 815 408 133 515 976 775 498 680 772 178 388 397 665 570 638 295 385 138 955 731 188
λ, hexadecimal	cbe45485dbfbb94cf26f94e441972b5d5dc8a1e7dd01dadf94d0592504ad52b0 cb06007077b9659b77427b1d4275155b5289bb1ef4656f8fd51cf8ee1c168c7b cf803f1598897281f8d566648c7e2ae7eec9b59db2cb468b06f57afbd180fbdd 34af9f0a6f79adbdcfa41e51c8aa757b16204390c35c1068e0bb2c186f15dcac 0f4e52b30580c1058a062a165602d328934bff2ed9cef44644c567d022de0c44 0e9975e0ae63b6d196feaef424d8ad74e29ebd6c94698eb09d678b1e8844b9cb 2c8c476bdf1423751c804174caa22ca82ab29a55d94c6592abfc4dfdc5769a10 57b019b35a211f8172f86ed7e0a31c0871d0d17760a049b07c4dafcf2e45e6c7 b2b436a905908f275b358252917f3a0029c854cde0df256f9c57dca11fc54602 31675c4119143858db6b5d1a57ed5915b50777c43639835b4d12197ce3dac0f4
λ^{-1}, decimal	88 197 048 086 653 553 990 193 318 000 083 285 420 073 593 027 008 419 646 778 994 813 808 463 880 326 977 971 906 648 018 154 508 812 529 199 357 974 875 578 370 882 992 036 788 104 258 782 948 436 766 055 840 795 823 707 866 720 155 354 530 034 095 685 559 171 753 740 106 477 861 834 465 556 544 573 469 239 965 451 694 440 137 579 904 003 250 416 043 478 898 505 851 535 460 530 306 879 634 909 352 779 399 127 389 132 451 357 261 828 913 965 845 690 995 492 095 810 780 370 058 799 773 656 975 446 398 466 825 129 137 636 112 539 297 619 655 123 932 542 512 504 890 386 877 727 816 825 677 138 388 366 741 346 984 168 460 616 709 186 801 106 564 005 641 880 392 417 282 093 099 444 975 568 209 336 642 003 405 761 219 603 395 184 402 465 417 392 013 297 579 883 333 264 778 060 207 709 925 411 584 621 962 266 104 265 870 820 067 804 499 391 149 414 982 726 589 854 374 201 829 523 332 207 964 319 691 641 794 637 377 704 126 551 921 107 779 545 327 367 265 181 821 702 025 304 062 686 536 635 617 705 843 010 814 431 740
a_1	18 065 580 261 771 683 859 731 951 611 087 020 321 789 588 578 978 316 593 278 149 140 449 584 700 074 766 446 620 345 461 505 266 312 776 955 237 290 075 888 986 680 828 789 932 383 804 193 253 867 215 213 026 904 511 948 160 654 160 972 693 817 909 415 557 520 210 393 225 949 269 148 423 511 216 676 352 585 131 298 293 450 533 744 504 326 193 109 601 016 770 301 837 128 519 887 307 675 134 137 476 551 644 194 871 490 896 634 138 634 216 834 196 644 033 211 302 087 092 923 151 635 056 591 001 719 966 137 840
b_1	-4 697 698 484 929 721 359 839 108 038 870 595 353 155 463 123 369 808 995 139 727 999 557 083 706 133 961 737 439 435 988 936 761 852 033 527 001 582 173 449 494 139 366 946 572 962 643 219 529 653 748 937 525 314 768 022 141 845 306 426 642 134 526 197 282 223 469 714 586 379 299 077 267 748 340 661 890 158 237 736 025 528 067 650 057 989 088 981 779 588 200 843 762 012 203 480 597 621 678 978 031 431 598 113 648 256 227 715 930 880 742 312 179 720 295 920 620 262 324 889 021 374 032 303 734 380 627 671 553

Item	Value
a_2	4 697 698 484 929 721 359 839 108 038 870 595 353 155 463 123 369 808 995 139 727 999 557 083 70 6 133 961 737 439 435 988 936 761 852 033 527 001 582 173 449 494 139 366 946 572 962 643 219 52 9 653 748 937 525 314 768 022 141 845 306 426 642 134 526 197 282 223 469 714 586 379 299 077 26 7 748 340 661 890 158 237 736 025 528 067 650 057 989 088 981 779 588 200 843 762 012 203 480 59 7 621 678 978 031 431 598 113 648 256 227 715 930 880 742 312 179 720 295 920 620 262 324 889 02 1 374 032 303 734 380 627 671 553
b_2	22 763 278 746 701 405 219 571 059 649 957 615 674 945 051 702 348 125 588 417 877 140 006 668 40 6 208 728 184 059 781 450 442 028 164 810 482 238 872 249 338 480 820 195 736 505 346 447 412 78 3 520 964 150 552 219 279 970 302 499 467 399 335 952 435 612 839 743 680 107 812 328 568 225 69 1 259 557 338 242 743 369 034 318 978 601 394 562 315 282 091 380 604 971 145 599 140 723 367 90 5 296 813 115 507 983 242 308 519 747 124 350 069 514 959 146 376 364 329 131 922 349 417 812 17 3 009 088 894 736 100 593 809 393

Table A6. The Cornacchia representation.

Item	Value
x, hexadecimal	523381535a2c9d98d8c2d2e0ac62bf3fbad5f551cff519b87e4a871593be45a8 c7086888e78a1b136834a4d16a4aaeef8bf1bee6102df15d1f315e0b4844c536 fa3562fa91c9c49abd108e4e8e53ba43cbaefb4a7c24082223db006fb59078aa 57453529a1c8d0fffe6161583d2821693352fe3d892d149dde83f0a639973359 90d06a5f8e3d8ce32a247cdba669928283b033df58121814391f10fae0aba1f8
x, bits	1279
y, hexadecimal	8bf996a554fce648f6fd81a0da78bba75bd93a131f0fb99396c882cc3489ee54 41a561613c8604eea27664efa9c13d887b37ec2f149a63b406e2ced290031e3f 249ff6dac6a6e79d7095dd1eac6e5616cda1f5c2882a1b8343aa184744f6ebc9 f5ea0c9acf3ddf61a700dffa56e3902ca5b76b2285869844bd5b7936ac441533 5c84887dac1851e77f7826690ffa365abfa4e789353c72eed67419c73f4e5bf9
y, bits	1280
Euclidean steps in the descent	746

Table A7. The twist.

Item	Value
d	2
b′	56
n_t, decimal	433 300 210 274 926 779 301 235 722 995 130 529 126 851 924 312 253 566 276 831 366 547 097 655 953 268 278 873 591 999 368 453 759 375 352 459 120 642 109 169 274 240 110 620 843 517 970 102 311 224 458 427 564 046 616 227 161 115 801 662 152 359 395 451 958 893 835 480 905 010 246 703 239 443 701 925 453 743 683 726 938 789 767 940 577 524 555 724 362 397 039 358 587 448 248 740 993 955 708 382 317 083 531 851 133 758 531 532 844 694 304 737 183 093 418 571 190 053 292 997 716 413 815 714 417 740 434 254 356 499 126 344 026 298 685 848 401 162 003 289 538 436 859 048 989 315 130 806 025 741 053 252 232 002 650 399 146 905 355 177 014 396 767 173 367 140 862 254 043 551 132 027 307 070 565 794 924 489 274 626 918 989 692 999 250 299 400 287 752 139 065 872 919 114 930 876 620 569 112 198 689 215 236 434 437 244 687 207 670 803 539 468 069 892 173 775 813 857 597 318 604 668 383 337 493 736 678 647 180 200 421 205 428 223 586 699 011 251 812 844 387 974 045 069 246 355 472 554 851 494 546 972 703 790 890 057 399
n_t, bits	2561
$v_3(n_t)$	1
rational 3-torsion, x	0
rational 3-torsion, y	335 761 661 411 349 378 960 702 279 347 687 552 651 399 078 831 781 695 583 484 122 076 187 083 157 816 673 418 673 201 754 616 705 056 007 562 213 526 874 459 558 746 074 396 781 192 530 742 508 388 172 918 153 279 410 897 255 193 919 747 881 974 247 814 340 960 136 166 768 737 113 563 959 545 907 214 431 149 438 935 371 844 453 137 111 425 289 011 042 547 023 228 462 075 556 663 162 059 973 288 893 409 042 612 244 292 557 227 916 055 223 555 731 804 967 197 554 712 488 519 340 911 717 332 584 152 277 093 288 836 151 802 970 479 478 150 330 874 310 716 836 733 448 648 564 003 376 689 113 938 250 128 567 443 562 781 767 692 335 320 332 039 014 161 393 196 508 045 610 765 071 962 795 352 467 419 929 294 836 051 037 414 544 413 831 580 565 950 319 810 750 995 889 663 459 438 063 474 429 987 236 352 660 738 150 989 355 212 027 981 855 013 782 867 815 883 850 872 868 482 270 205 110 851 728 688 056 807 180 669 878 751 195 751 830 412 291 112 468 843 303 080 472 704 610 581 425 793 691 952 723 617 845 587 077 369 162

Appendix B. The verification listings

These are the routines behind the tables. They are given in the form in which they were executed, so that the report can be reproduced without a black box.

Listing B1. Cornacchia's algorithm for p = x^2 + 3y^2.

```
def cornacchia(p):
    """Solve p = x^2 + 3y^2 for p = 3 (mod 4)."""
    s = pow((-3) % p, (p + 1) // 4, p)      # sqrt(-3) mod p
    r0, r1 = p, s
    while r1 * r1 > p:                      # Euclidean descent
        r0, r1 = r1, r0 % r1
    x = r1
    y = isqrt((p - x * x) // 3)
    assert x * x + 3 * y * y == p
    return x, y
```

The descent for this prime takes 746 steps and yields x of 1279 bits and y of 1280 bits.

Listing B2. The published decomposition routine, and what it does.

```
# the routine as published, verbatim
def gauss_reduce((N, 0), (-lam % N, 1)):
    ...
def decompose(k):
    c1 = round(b2 * k / det)
    c2 = round(-b1 * k / det)
    return k - c1 * a1 - c2 * a2, -(c1 * b1 + c2 * b2)

# what actually happens on a 2560-bit scalar
k = 1 << 2559 | 0x9E3779B97F4A7C15
c1 = round(b2 * k / det)
#  OverflowError: integer division result too large for a float
#
# because  b2 * k  is about 2**3838,  det  is about 2**2560,
# so the quotient is about 2**1278 -- and a double cannot hold
# a value larger than about 2**1024.
```

The analysis of the failure is given in Section 13, item 1; the corrected routine is Listing 1.

Listing B3. The Crandall reduction.

```
def crandall_reduce(x, p, BITS=2560, HALF=320, c=132200237):
    """Reduce x modulo p = 2**BITS - 2**HALF - c."""
    mask = (1 << HALF) - 1
    while x.bit_length() > HALF:
        hi = x >> HALF
        lo = x & mask
        x = lo + hi * c                    # 2**2560 == 2**320 + c  (mod p)
    return x % p
```

The identity used is a congruence, 2^2560 = 2^320 + c (mod p), not an equality. Each fold multiplies a 320-bit quantity by the 32-bit constant c.

Listing B4. The sextic order technique.

```
def sextic_orders(p, x, y):
    """The six candidate traces and the six candidate orders."""
    out = []
    for t in (x, x + 3 * y, x - 3 * y):
        for sign in (+1, -1):
            tr = sign * t
            out.append((tr, p + 1 - tr))
    return out

def certify_order(model, candidate, tests=32, seed=12345):
    """Candidate is the order of `model` iff every random point has that
    order and the candidate lies inside the Hasse window."""
    if not (hasse_lo(p) <= candidate <= hasse_hi(p)):
        return False
    for _ in range(tests):
        R = find_point(model, seed=random())
        if mul(candidate, R, model) is not None:
            return False
    return True
```

The certification test is the same one used in Section 4: a candidate order is accepted only if it annihilates random points of the model and lies inside the Hasse window.

Listing B5. The base-point ritual.

```
def base_point(p):
    """The ritual: the first x >= 0 for which x^3 + 7 is a residue,
    taken with the even y, then doubled."""
    x = 0
    while True:
        y = sqrt_mod(pow(x, 3, p) + 7, p)
        if y is not None:
            break
        x += 1
    P0 = (x, y if y % 2 == 0 else p - y)
    return P0, double(P0)
```

For this prime the loop exits at x = 2 immediately, because 15 is a quadratic residue.

Appendix C. Reproduction

All results in this report are produced by the scripts listed below, run in the directory that contains them. The arithmetic is pure Python and requires no external packages; the primality tests are Miller–Rabin and the modular square root exploits $p \equiv 3 \pmod 4$.

Table C1. Artifacts and runtimes.

Artifact	Purpose	Runtime
13_paper_data.py	parameters, order certification, sextic spectrum, twist, endomorphism, embedding degree, benchmark, 256-bit recipe	about 33 s
14_figures.py	the five principal figures	about 2 s
19_figures_extra.py	the ritual scan, the security landscape, the decomposition lengths	about 16 s
20_extra_data2.py	the sextic quotient, the six model certifications, torsion, Cornacchia statistics	about 38 s
21_ceiling_data.py	the security ceiling, the cost model and the checked field reduction	about 2 s
22_figure_ceiling.py	the ceiling figure	about 3 s
23_renumber_figures.py	the renumbering of the figure captions	under 1 s
24_small_factors_analysis.py	the small-factor sieve to 2e7 and the Eisenstein-norm analysis	about 40 s
25_generate_curve.py	the generation program printed in full in Appendix E	0.1 s at 256 bits, 25.4 s at 2560 bits
26_split_code.py	the splitting of the generation program into printable listings	under 1 s
27_add_citations.py	the insertion of the citation markers	under 1 s
29_minimality_sieve.py	the full-scale sieve over c at 2560 bits, Section 10.5	about 25 min
30_minimality_class.py	the sextic-class identification of the 52 surviving candidates	about 5 min
32_classify_hits.py	the classification of the hits and the certification of the first candidate of the class b = 7	about 4 min
34_minimality_tables.py	the twist-order analysis of the fourteen candidates of the class b = 7	about 2 min
35_class3_check.py	the sextic-quotient analysis of the 52 surviving primes	about 1 min
pap_common.py, pap_body_a.py, pap_body_b.py, pap_body_c.py, pap_body_d.py, build_paper_v2.py	the assembly of this document	under 2 s
render_v2.py	the rendering to PDF and the page audit	about 12 s

The figures are vector SVG, so the document remains sharp at any magnification and the tables are generated from the same data files that produced the numbers.

Integrity of the generation program printed in Appendix E: the shipped file `25_generate_curve.py` has SHA-256 checksum `a42e490124cadf1bf06396c9ca5e54744ec96e9bcd17b9b9eb94518e899d9137`. This is a checksum of a program file and nothing else: it is quoted so that a reader can confirm that the program printed in Appendix E is the very program that produced the numbers in this document. No cryptographic hash function is specified, used or certified anywhere in this document; the scope statement of Section 1 and of the colophon is unaffected by it.

Two methodological notes deserve to be repeated here. First, the elliptic-curve arithmetic used for this report was written independently of the originating code and was validated against secp256k1, where the group order and the generator are known in closed form; a known-answer test of that kind is what caught an error in the point-addition routine during development. Second, no unbounded factorisation was attempted at any point: the small-factor tables of Section 9.2 come from trial division with a bound, and the residual cofactors are reported as uncomputed rather than guessed at.

Appendix D. The security ceiling and the cost of the field

This appendix collects, in one place, the two statements that Section 9.9 makes in prose: what the security ceiling of the curve is, and what it costs to carry it. The ceiling is a property of the group order alone. Since $E(F_p)$ is cyclic of prime order N with 2560 bits, the generic discrete logarithm costs $\approx \sqrt{N} = 2^{1280}$ group operations, and the question the table below answers is: which other attacks could cost less, and what certified property of this curve stops them?

Table D1. Routes to the discrete logarithm, and the certified obstruction that closes each one.

Route	What it would require	Status	Established in
Generic discrete logarithm (Pollard rho)	requires sqrt(N) group operations	the ceiling itself	Section 9.1
Pohlig-Hellman (smooth order)	requires N to be composite with small factors	closed: N is prime	Section 4, Section 9.2
MOV / Frey-Ruck pairing reduction	requires a small embedding degree ord_N(p)	closed: ord_N(p) > 400000	Section 9.3
Weil descent / GHS	requires rational 2- or 3-torsion and a genus-2/3 lift	closed: no rational 2- or 3-torsion; the curve is ordinary	Section 7
Anomalous curve (Smart, Satoh-Araki)	requires N = p	closed: N is far below p	Section 9.5
Trace-zero 2-dimensional lattice attack	requires a supersingular Koblitz curve with a rank-4 quaternion ring and a non-public Frobenius	closed: the curve is ordinary, t != 0, End_Fp(E) = Z[w] has rank 2, and lambda is public	Section 8.6, Section 9.5
Twist transfer (small subgroup / anomalous twist)	requires a shared factor between N and the twist order	closed: gcd(N, n_t) = 1	Section 6
Special-purpose index calculus for this family	not a theorem; would have to beat sqrt(N) asymptotically	statement of the art, not a certified closure	Section 9.1
Invalid curve / small subgroup in the protocol layer	not a property of the group	out of scope: this document certifies the curve, not a protocol	Section 14.8

The only route that reaches the group is the generic one; every structural route is closed by an obstruction certified earlier in this document.

Three of these deserve a comment. The **trace-zero two-dimensional lattice attack** is the route that genuinely damages other curves in the Koblitz family: on a supersingular curve of trace zero the endomorphism ring has rank four and the Frobenius is not public, so the group can be attacked with a two-dimensional lattice at roughly the square root of a square root. Section 8 established that this curve is ordinary, that $\mathrm{End}_{F_p}(E) = \mathbb{Z}[\omega]$ has rank two, that the trace is non-zero and that the endomorphism is public scalar multiplication by λ; each of those four facts removes one precondition of that attack. The **twist transfer** route would move a discrete logarithm from the quadratic twist back to E; Section 6 established that $\gcd(N, n_t) = 1$, which makes the transfer impossible regardless of how the twist's cofactor factors. The **special-purpose index calculus** row is not a closure: it is a statement of the art, and the honest position is that no index-calculus attack against this curve is known and none is excluded by the properties certified here.

The cost side is a table of ordinary arithmetic. Nothing in it is a weakness; it is the invoice for the ceiling.

Table D2. The cost of carrying the ceiling.

Quantity	This curve	secp256k1	Ratio
Field element	320 bytes	32 bytes	10×
Scalar	320 bytes	32 bytes	10×
Compressed point	321 bytes	33 bytes	≈10×
Uncompressed point	641 bytes	65 bytes	≈10×
Security ceiling	1280 bits	128 bits	≈10× in bits, 1152 bits of margin
Reduction of one product	2 folds, 2 constant multiplications	no comparable structure	—
Scalar multiplication	joint GLV, 1280 D + 418 A	wNAF, 2561 D + 430 A	1.548× faster
Effect on the ceiling	none	none	—

The last row is the point of the table: the endomorphism and the Crandall form are engineering economies. Neither of them changes what an attacker has to do.

The reduction row was measured rather than asserted. A 5120-bit product is reduced by the relation $2^{2560} = 2^{320} + c \pmod{p}$: the high part is folded down 2 times, each fold costing one multiplication by the 32-bit constant c and one shift-and-add, and the residue is then reduced by at most 0 subtraction(s) of p. The implementation was checked against exact division on 200 random 2560-bit products and agreed in every case; the largest residue observed was 2560 bits, inside the modulus.

Table D3. The landscape, in one line each.

Curve	Source	Field bits	Security ceiling (bits)	Bytes per element
NIST P-192 (secp192r1)	NIST FIPS 186-4	192	96	24
secp256k1	SEC 2	256	128	32
NIST P-384 (secp384r1)	NIST FIPS 186-4	384	192	48
secp521r1 (NIST P-521)	SEC 2	521	260	66
secp2560k1 ●	this document	2560	1280	320

The marked entry is this curve. The ceiling column is the generic discrete logarithm work factor, that is the square root of the field size, and not a strength figure quoted from a standard; the byte column is the price of one field element.

Every comparator in this landscape is a published, standardised curve: secp192r1 and secp384r1 are NIST P-192 and NIST P-384 of NIST FIPS 186-4 and are also listed in SEC 2; secp256k1 and secp521r1 are listed in SEC 2. The ceiling column is the generic discrete-logarithm work factor, that is the square root of the field size, and not a strength figure drawn from a standard.

No standardised prime-field Koblitz curve has a field larger than 256 bits: the largest Koblitz curve in SEC 2 is secp256k1, and NIST SP 800-186 lists none over a prime field. The curve of this document is therefore the first large prime-field Koblitz curve to be documented with a full parameter set and a reproducible generation procedure.

It is worth stating the obvious consequence, since the table makes it visible: the security level rises by a factor of about 1152 bits while the cost per operation rises by a factor of ten in bandwidth and about a hundred in storage. That is the whole trade, and Section 9.8 gives the honest reason for accepting it.

Appendix E. The generation program in full

This appendix prints the program that generates and certifies the curve, in its entirety, so that the document can be checked without trusting the author. The program is `25_generate_curve.py`, shipped alongside this document; the listings below are its source, split into 42-line blocks so that no listing is broken across a

page. Concatenating the blocks reproduces the shipped file byte for byte, and the build script that produced this appendix checks that reconstruction before it emits the page.

The program is written in plain Python with no third-party package. It carries its own integer arithmetic, its own curve arithmetic and its own primality tests, and it therefore depends on nothing except an interpreter. A checksum over the shipped file, and the commands that reproduce it, are recorded in Appendix C. The file is 597 lines and 18013 bytes long, and the reconstruction check performed by the build script — that concatenating the blocks below reproduces the shipped file byte for byte — is recorded as **passed**.

Table E1. What each part of the program certifies, and where the paper discusses it.

Listing	Part of the program	What it establishes	Discussed in
E1	Header, command-line interface, imports	The program is self-contained: no third-party package is used, so that the reproduction depends on nothing but a Python interpreter.	§10, Appendix C
E2–E4	Integer arithmetic	Primality by Miller–Rabin [11,12]; square roots by the $p \equiv 3 \pmod 4$ fast path and Tonelli–Shanks otherwise [14]; cube roots of unity; exact nearest-integer rounding by division only, which replaces the floating-point rounding rejected in Section 13.	§2, §8, §13
E5–E6	Curve arithmetic	Jacobian coordinates, so that point addition never needs an inversion; the routines are the same ones used by every other program in this study and are validated against the certified secp256k1 constants.	§3, §4
E7	The search	The Crandall form $2^{2560} - 2^{320} - c$ [13], the sieve over $c \equiv 5 \pmod{12}$, and Cornacchia's algorithm [8,9] for the representation of the field prime as an Eisenstein norm [6,7].	§4, §5, §10
E8–E9	The sextic family	The six models $b = 7 \cdot 3^j$ and their traces, followed by the certification of each candidate order: points of that model annihilated by the candidate, and the candidate shown to lie inside that model's Hasse window [3].	§5, §12
E10	The base-point ritual	The first x at or above the seed for which $x^3 + b$ is a quadratic residue, the even square root, and the doubling that produces the published generator; the closed form $x(G) \equiv (p - 8)/5$ is checked when the seed abscissa is 2.	§3
E11–E12	The endomorphism and the CM lattice	The cubic endomorphism, the relation $\lambda^2 + \lambda + 1 \equiv 0$ modulo the order, the two-dimensional lattice by exact Gauss reduction, and the exact integer decomposition of a scalar [29,30].	§8
E13–E18	The driver	The nine stages of the recipe in order, the quadratic twist and its coprimality check, the Crandall reduction of a product, and the certification of the order. Every stage is verified before the next is started.	§4, §6, §10, §12

The mapping is deliberately coarse: the program is a single instrument, and the paper is its commentary.

E.1 Running the program

Two invocations reproduce everything claimed in this document. The first searches, at 256 bits, and finds the original secp256k1 curve of the recipe; the second verifies the 2560-bit instance of this document. Both accept `--out` and write a JSON file containing every quantity the program computed.

Table E2. The two invocations, the search each performs and the time each takes.

Command	What it does	Scope	Wall-clock
`python -u 25_generate_curve.py --bits 256 --cap 1200 --seed 2 --out gen256.json`	Search at 256 bits over c ≡ 5 (mod 12) with c ≤ 1200. Finds the single admissible value c = 977, whose unique prime sextic order is that of secp256k1 and whose trace is that of secp256k1.	82 candidates tested	0.1 s
`python -u 25_generate_curve.py --bits 2560 --c 132200237 --seed 2 --out gen2560.json`	Verification of the 2560-bit instance of this document: primality of the field, Cornacchia representation, the six sextic orders, the base-point ritual, the endomorphism, the CM lattice, the quadratic twist, the certification of the order and the check of the field reduction.	one instance, no search	25.4 s

The times are from the verification run of this document on the machine described in Appendix C. They are indicative of the order of magnitude only.

The output of the second invocation is the file that the rest of this document is built from: the field, the order, the trace, the Cornacchia coordinates, the six sextic models and their certified orders, the generator, the endomorphism, the lattice basis, the twist and the certification flags are all written to it in one pass, and the tables of Sections 2 to 9 are transcriptions of that file.

E.2 The program

Listing E1. The program header, its command-line interface and its imports

```
#!/usr/bin/env python3
"""
secp2560k1 -- complete, self-contained generation and certification program.

The program rebuilds the curve from the field prime alone and certifies every
property the verification report prints.  It is pure Python and needs no
third-party package.

    stage 1   the prescribed family   p = 2^bits - 2^(bits/8) - c,   c = 5 (mod 12)
    stage 2   certify that p is prime
    stage 3   Cornacchia:  p = x^2 + 3y^2
    stage 4   the six sextic models b = 7 * 3^j and their six candidate orders
    stage 5   certify that exactly one of the six has prime order; that one is the curve
    stage 6   the base-point ritual
    stage 7   the cubic endomorphism: omega, lambda, and the CM lattice
    stage 8   the quadratic twist
    stage 9   certification: N*G = O, the Hasse window, the cofactor

Usage
    python 25_generate_curve.py --bits 256                      # full search
    python 25_generate_curve.py --bits 2560 --c 132200237       # verify this curve

The first invocation performs the whole minimality search and reproduces
secp256k1 exactly.  The second skips the search (which at 2560 bits is out of
reach of an interpreter) and verifies the parameters of secp2560k1 instead.
"""

import argparse
import json
import os
import random
import sys
import time
```

Listing E2. Integer arithmetic: primality, square roots, cube roots of unity, exact rounding (part 1 of 3)

```
# ---- PART: arithmetic ----------------------------------------------------
# Integer helpers, primality, modular square root and modular cube roots.


def log(msg):
    print(msg, flush=True)


def is_prime(n, rounds=40, seed=20251022):
    """Miller-Rabin with a fixed seed so that the verdict is reproducible."""
    if n < 2:
        return False
    for q in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % q == 0:
            return n == q
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    rng = random.Random(seed)
    for _ in range(rounds):
        a = rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True


def sqrt_mod(a, p):
    """A square root of a modulo the odd prime p, or None.  Tonelli-Shanks."""
    a %= p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
```

Listing E3. Integer arithmetic: primality, square roots, cube roots of unity, exact rounding (part 2 of 3)

```
        return pow(a, (p + 1) // 4, p)
    q, s = p - 1, 0
    while q % 2 == 0:
        q //= 2
        s += 1
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    m, c, t, r = s, pow(z, q, p), pow(a, q, p), pow(a, (q + 1) // 2, p)
    while t != 1:
        i, t2 = 0, t
        while t2 != 1:
            t2 = t2 * t2 % p
            i += 1
        b = pow(c, 1 << (m - i - 1), p)
        m, c, t, r = i, b * b % p, t * b * b % p, r * b % p
    return r


def cube_root_of_unity(p):
    """The two roots of x^2 + x + 1 modulo a prime p = 1 (mod 3)."""
    s = sqrt_mod(p - 3, p)
    if s is None:
        return None
    inv2 = pow(2, -1, p)
    z1 = ((-1 + s) * inv2) % p
    z2 = ((-1 - s) * inv2) % p
    assert (z1 * z1 + z1 + 1) % p == 0 and (z2 * z2 + z2 + 1) % p == 0
    return z1, z2


def round_div(a, b):
    """Round a/b to the nearest integer, ties to even, using integers only.

    b must be positive.  This is the routine that the published listing gets
    wrong: a floating-point division of two 2560-bit integers does not fit in
    a double, and even when it fits the 53-bit mantissa silently truncates the
    low order digits of the quotient.
    """
    q, r = divmod(a, b)
    twice = 2 * r
    if twice > b:
```

Listing E4. Integer arithmetic: primality, square roots, cube roots of unity, exact rounding (part 3 of 3)

```
        q += 1
    elif twice == b and (q & 1):
        q += 1
    return q


def isqrt(n):
    import math
    return math.isqrt(n)
```

Listing E5. Curve arithmetic in Jacobian coordinates (part 1 of 2)

```
# ---- PART: curve arithmetic ------------------------------------------------
# Jacobian coordinates for y^2 = x^3 + a*x + b with a = 0, b = 7.


def jac_double(X1, Y1, Z1, p):
    if Y1 == 0 or Z1 == 0:
        return (0, 0, 0)
    A = Y1 * Y1 % p
    B = 4 * X1 * A % p
    C = 8 * A * A % p
    D = 3 * X1 * X1 % p
    X3 = (D * D - 2 * B) % p
    Y3 = (D * (B - X3) - C) % p
    Z3 = 2 * Y1 * Z1 % p
    return (X3, Y3, Z3)


def jac_add(X1, Y1, Z1, X2, Y2, Z2, p):
    if Z1 == 0:
        return (X2, Y2, Z2)
    if Z2 == 0:
        return (X1, Y1, Z1)
    Z1Z1 = Z1 * Z1 % p
    Z2Z2 = Z2 * Z2 % p
    U1 = X1 * Z2Z2 % p
    U2 = X2 * Z1Z1 % p
    S1 = Y1 * Z2 * Z2Z2 % p
    S2 = Y2 * Z1 * Z1Z1 % p
    if U1 == U2:
        if S1 != S2:
            return (0, 0, 0)
        return jac_double(X1, Y1, Z1, p)
    H = (U2 - U1) % p
    I = 2 * H % p
    I = I * I % p
    J = H * I % p
    r = 2 * (S2 - S1) % p
    V = U1 * I % p
    X3 = (r * r - J - 2 * V) % p
    Y3 = (r * (V - X3) - 2 * S1 * J) % p
    Z3 = ((Z1 + Z2) ** 2 - Z1Z1 - Z2Z2) % p
    Z3 = Z3 * H % p
```

Listing E6. Curve arithmetic in Jacobian coordinates (part 2 of 2)

```
    return (X3, Y3, Z3)


def to_affine(X, Y, Z, p):
    if Z == 0:
        return None
    zi = pow(Z, -1, p)
    zi2 = zi * zi % p
    return (X * zi2 % p, Y * zi2 % p * zi % p)


def mul(k, pt, p, b=7):
    """k * pt in affine coordinates; None for the point at infinity."""
    if pt is None or k == 0:
        return None
    X, Y, Z = pt[0], pt[1], 1
    acc = (0, 0, 0)
    for bit in bin(k)[2:]:
        acc = jac_double(*acc, p)
        if bit == "1":
            acc = jac_add(acc[0], acc[1], acc[2], X, Y, Z, p)
    return to_affine(*acc, p)


def on_curve(pt, p, b=7):
    if pt is None:
        return False
    x, y = pt
    return (y * y - x * x * x - b) % p == 0


def lift_x(x, p, b=7):
    y = sqrt_mod(x * x * x + b, p)
    if y is None:
        return None
    return (x, y)
```

Listing E7. The field prime: the Crandall form, the sieve over c, Cornacchia's algorithm

```
# ---- PART: the search -----------------------------------------------------
# The prescribed family and the sieve over c = 5 (mod 12).


def make_prime(bits, c):
    """p = 2^bits - 2^(bits/8) - c."""
    return (1 << bits) - (1 << (bits // 8)) - c


def sieve_candidates(bits, limit):
    """c = 5 (mod 12), ascending."""
    c = 5
    while c <= limit:
        yield c
        c += 12


def cornacchia(p):
    """p = x^2 + 3y^2 for p = 1 (mod 3) and p = 3 (mod 4)."""
    s = sqrt_mod(p - 3, p)
    if s is None:
        return None
    r0, r1 = p, s
    while r1 * r1 > p:
        r0, r1 = r1, r0 % r1
    x = r1
    y = isqrt((p - x * x) // 3)
    if x * x + 3 * y * y != p:
        return None
    return x, y
```

Listing E8. The six sextic models and the certification of their orders

```
# ---- PART: the sextic family ------------------------------------------------
# Six models b = 7*3^j, their traces and their candidate orders.


MODELS = [
    (0, 7, lambda x, y: x + 3 * y),
    (1, 21, lambda x, y: 2 * x),
    (2, 63, lambda x, y: x - 3 * y),
    (3, 189, lambda x, y: -(x + 3 * y)),
    (4, 567, lambda x, y: -2 * x),
    (5, 1701, lambda x, y: -(x - 3 * y)),
]


def sextic_spectrum(p, x, y):
    """The six (model, trace, candidate order) triples of the j = 0 family."""
    out = []
    for j, b, trace_of in MODELS:
        t = trace_of(x, y)
        out.append((j, b, t, p + 1 - t))
    return out


def certify_order(b_model, candidate, p, tests=12, seed=99):
    """candidate is the order of the model iff it annihilates random points
    of that model and lies inside the Hasse window."""
    lo = p + 1 - 2 * isqrt(p)
    hi = p + 1 + 2 * isqrt(p)
    if not (lo <= candidate <= hi):
        return False
    rng = random.Random(seed)
    x = 0
    found = 0
    while found < tests:
        x += 1
        pt = lift_x(x, p, b_model)
        if pt is None:
            continue
        found += 1
        if mul(candidate, pt, p, b_model) is not None:
            return False
    return True
```

Listing E9. The base-point ritual

```
# ---- PART: the base point ------------------------------------------------
# The ritual: the first x for which x^3 + 7 is a residue, taken even, doubled.


def base_point(p, b=7, seed=0):
    """The ritual: the first x >= seed for which x^3 + b is a quadratic
    residue, taken with the even y, then doubled."""
    x = seed
    while True:
        pt = lift_x(x, p, b)
        if pt is not None:
            y = pt[1]
            if y % 2:
                y = p - y
            P0 = (x, y)
            G = mul(2, P0, p, b)
            return P0, G
        x += 1


def canonical_secp256k1_generator():
    """The generator published for secp256k1, used only as an extra check."""
    return (0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
            0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8)
```

Listing E10. The endomorphism, the CM lattice and the exact decomposition (part 1 of 2)

```
# ---- PART: the endomorphism and the CM lattice ----------------------------
# omega, lambda, the lattice basis, and the exact integer decomposition.


def gauss_reduce(u, v):
    """Two-dimensional Lagrange reduction on integer vectors."""
    def norm(w):
        return w[0] * w[0] + w[1] * w[1]

    def dot(a, b):
        return a[0] * b[0] + a[1] * b[1]

    while True:
        if norm(u) > norm(v):
            u, v = v, u
        q = round_div(dot(u, v), norm(u))
        if q == 0:
            return u, v
        v = (v[0] - q * u[0], v[1] - q * u[1])


def cm_lattice(n, lam):
    """A reduced basis of {(a, b) : a + b*lambda = 0 (mod n)}."""
    u, v = gauss_reduce((n, 0), ((-lam) % n, 1))
    return u, v


def decompose(k, a1, b1, a2, b2, det, n, lam):
    """k = k1 + k2*lambda (mod n) with |k1|, |k2| about sqrt(n), integers only."""
    c1 = round_div(b2 * k, det)
    c2 = round_div(-b1 * k, det)
    k1 = k - c1 * a1 - c2 * a2
    k2 = -(c1 * b1 + c2 * b2)
    return k1, k2


def crandall_reduce(x, p, bits, half, c):
    """Reduce x modulo p = 2^bits - 2^half - c.

    Since 2^bits = 2^half + c (mod p), a high part hi = x >> bits is
    replaced by hi * (2^half + c): one multiplication by the 32-bit constant
    c and one shift-and-add.  Folding continues until the value fits in
```

Listing E11. The endomorphism, the CM lattice and the exact decomposition (part 2 of 2)

```
    `bits` bits; at most one subtraction of p then remains.
    """
    M = 1 << bits
    mask = M - 1
    guard = 0
    while x >= M:
        hi = x >> bits
        x = (x & mask) + hi * ((1 << half) + c)
        guard += 1
        if guard > 16:
            raise RuntimeError("reduction did not terminate")
    while x >= p:
        x -= p
    return x
```

Listing E12. The driver: the stages, the twist and the command line (part 1 of 6)

```
# ---- PART: the driver ----------------------------------------------------
# The pipeline, stage by stage, and the report it prints.


def twist_of(p, t, b=7):
    """The quadratic twist: its model, its order and its rational 3-torsion."""
    d = 2
    while pow(d, (p - 1) // 2, p) != p - 1:
        d += 1
    b_twist = b * d * d * d % p
    n_t = p + 1 + t
    return d, b_twist, n_t


def search(bits, cap, seed):
    """Stage 1 to 5: walk c = 5 (mod 12) until the recipe fires."""
    tried = 0
    for c in sieve_candidates(bits, cap):
        tried += 1
        p = make_prime(bits, c)
        if not is_prime(p):
            continue
        xy = cornacchia(p)
        if xy is None:
            continue
        x, y = xy
        spec = sextic_spectrum(p, x, y)
        primes = [(j, bb, tr, o) for (j, bb, tr, o) in spec if is_prime(o)]
        if len(primes) != 1:
            continue
        j, bb, tr, o = primes[0]
        if j != 0:
            continue
        log("  hit: c = %d, p prime, exactly one prime sextic order, model b = 7" % c)
        return c, p, x, y, spec, tried
    return None, None, None, None, None, tried


def analyse(bits, c, spec=None, seed=2):
    """Every stage from the prime to the certification."""
    out = {"bits": bits, "c": c}
    p = make_prime(bits, c)
```

Listing E13. The driver: the stages, the twist and the command line (part 2 of 6)

```
    out["p"] = hex(p)
    out["p_dec"] = str(p)
    log("  p = 2^%d - 2^%d - %d" % (bits, bits // 8, c))
    log("  p bits = %d, p mod 12 = %d" % (p.bit_length(), p % 12))

    log("  stage 2: primality of p")
    assert is_prime(p)
    out["p_is_prime"] = True

    log("  stage 3: Cornacchia")
    x, y = cornacchia(p)
    out["cornacchia_x"] = hex(x)
    out["cornacchia_y"] = hex(y)
    out["cornacchia_x_bits"] = x.bit_length()
    out["cornacchia_y_bits"] = y.bit_length()
    log("    x = %d bits, y = %d bits" % (x.bit_length(), y.bit_length()))

    log("  stage 4: the sextic spectrum")
    if spec is None:
        spec = sextic_spectrum(p, x, y)
    models = []
    for (j, bb, tr, o) in spec:
        models.append({
            "j": j, "b": bb, "trace": hex(tr), "order": hex(o),
            "order_bits": o.bit_length(), "order_is_prime": is_prime(o),
        })
        log("    j = %d, b = %-5d order bits %d prime %s"
            % (j, bb, o.bit_length(), models[-1]["order_is_prime"]))
    out["models"] = models
    prime_models = [m for m in models if m["order_is_prime"]]
    assert len(prime_models) == 1, "the recipe requires exactly one prime order"
    assert prime_models[0]["j"] == 0, "the prime order must be the model b = 7"

    n = int(prime_models[0]["order"], 16)
    t = p + 1 - n
    out["n"] = hex(n)
    out["n_dec"] = str(n)
    out["n_bits"] = n.bit_length()
    out["t"] = hex(t)
    out["t_bits"] = t.bit_length()
    assert is_prime(n)
    log("    N = %d bits, prime" % n.bit_length())
```

Listing E14. The driver: the stages, the twist and the command line (part 3 of 6)

```
    log("    certifying the order of the model b = 7 by point annihilation")
    ok = certify_order(7, n, p, tests=6)
    out["order_certified_by_points"] = ok
    assert ok, "the candidate order did not annihilate the test points"
    log("    the order is certified: it annihilates random points of the model")

    log("  stage 6: the base-point ritual")
    P0, G = base_point(p, seed=seed)
    assert on_curve(G, p)
    out["Gx"] = hex(G[0])
    out["Gy"] = hex(G[1])
    out["P0_x"] = str(P0[0])
    out["P0_y_even"] = (P0[1] % 2 == 0)
    out["G_is_2P0"] = (mul(2, P0, p) == G)
    log("    P0 = (%d, even), G = 2*P0, on curve" % P0[0])
    # the closed form of the doubled abscissa for the seed x0 = 2
    if P0[0] == 2:
        closed = ((p - 8) // 5) % p
        out["Gx_closed_form_holds"] = (G[0] == closed)
        assert G[0] == closed
        log("    x(G) = (p - 8)/5 (mod p), as the closed form predicts")

    log("  stage 7: the cubic endomorphism")
    z1, z2 = cube_root_of_unity(p)
    omega = None
    lam = None
    for z in (z1, z2):
        phiG = (z * G[0] % p, G[1])
        for cand in cube_root_of_unity(n):
            if mul(cand, G, p) == phiG:
                omega, lam = z, cand
    assert omega is not None, "the endomorphism did not match"
    assert (lam * lam + lam + 1) % n == 0
    out["omega"] = hex(omega)
    out["lambda"] = hex(lam)
    out["lambda_inv"] = hex(pow(lam, -1, n))
    log("    omega found, lambda = %d bits, lambda^2 + lambda + 1 = 0 (mod N)" % lam.bit_length())

    u1, u2 = cm_lattice(n, lam)
    a1, b1 = u1
    a2, b2 = u2
```

Listing E15. The driver: the stages, the twist and the command line (part 4 of 6)

```
    det = a1 * b2 - a2 * b1
    assert det == n or det == -n
    assert (a1 + b1 * lam) % n == 0 and (a2 + b2 * lam) % n == 0
    out["basis"] = {"a1": str(a1), "b1": str(b1), "a2": str(a2), "b2": str(b2),
                    "det": str(det), "det_equals_n": det == n}
    log("    CM lattice basis, det = N exactly, both relations hold")

    rng = random.Random(7)
    worst = 0
    for _ in range(20):
        k = rng.randrange(1 << (bits - 1), 1 << bits)
        k1, k2 = decompose(k, a1, b1, a2, b2, det, n, lam)
        assert (k1 + k2 * lam - k) % n == 0
        worst = max(worst, max(abs(k1).bit_length(), abs(k2).bit_length()))
    out["decompose_max_half_bits"] = worst
    log("    exact integer decomposition verified, worst half = %d bits" % worst)

    log("  stage 8: the quadratic twist")
    d, b_twist, n_t = twist_of(p, t)
    pt3 = lift_x(0, p, b_twist)
    has3 = False
    if pt3 is not None:
        has3 = (mul(3, pt3, p, b_twist) is None)
    out["twist"] = {"d": d, "b": str(b_twist), "order": hex(n_t),
                    "order_bits": n_t.bit_length(),
                    "v3": _v3(n_t),
                    "rational_3torsion": has3,
                    "gcd_with_n": _gcd(n, n_t)}
    log("    d = %d, b' = %d, n_t = %d bits, gcd(N, n_t) = %d"
        % (d, b_twist, n_t.bit_length(), _gcd(n, n_t)))

    log("  stage 9: certification")
    assert mul(n, G, p) is None, "N*G is not the point at infinity"
    lo = p + 1 - 2 * isqrt(p)
    hi = p + 1 + 2 * isqrt(p)
    assert lo <= n <= hi
    out["N_times_G_is_O"] = True
    out["n_in_hasse_window"] = True
    out["cofactor"] = 1
    log("    N*G = O and N lies in the Hasse window, so #E(Fp) = N and h = 1")

    log("  reduction check")
```

Listing E16. The driver: the stages, the twist and the command line (part 5 of 6)

```
    prod = (G[0] * G[0]) % (1 << (2 * bits))
    red = crandall_reduce(prod, p, bits, bits // 8, c)
    assert red == prod % p
    out["crandall_ok"] = True
    log("    the Crandall reduction agrees with exact division")
    return out


def _gcd(a, b):
    while b:
        a, b = b, a % b
    return a


def _v3(n):
    """The exponent of 3 in n."""
    e = 0
    while n % 3 == 0:
        n //= 3
        e += 1
    return e


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--bits", type=int, default=256)
    ap.add_argument("--c", type=int, default=None)
    ap.add_argument("--cap", type=int, default=4000)
    ap.add_argument("--seed", type=int, default=2,
                    help="the first abscissa tried by the base-point ritual")
    ap.add_argument("--out", default=None)
    args = ap.parse_args()

    t0 = time.time()
    log("secp2560k1 generator, bits = %d" % args.bits)
    if args.c is None:
        log("stage 1: searching c = 5 (mod 12)")
        c, p, x, y, spec, tried = search(args.bits, args.cap, args.seed)
        if c is None:
            log("no hit within the cap")
            return 1
        log("  candidates tested: %d" % tried)
```

Listing E17. The driver: the stages, the twist and the command line (part 6 of 6)

```
        out = analyse(args.bits, c, spec, args.seed)
        out["candidates_tested"] = tried
    else:
        out = analyse(args.bits, args.c, None, args.seed)
        out["candidates_tested"] = None
        out["search_performed"] = False
    out["elapsed_seconds"] = round(time.time() - t0, 2)
    log("done in %.1f s" % out["elapsed_seconds"])
    if args.out:
        with open(args.out, "w", encoding="utf-8") as f:
            json.dump(out, f, indent=1)
        log("wrote " + args.out)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

End of the program. The complete file is 597 lines and 18,013 bytes; the checksum of the shipped file is recorded in Appendix C.

References

The statements made in this document are either proved inside it or certified by the programs listed in Appendix C. Where a statement rests on a result that is classical rather than elementary, the result is cited here and marked in the text with a bracketed number in the form [n]. Nothing outside this list has been relied upon.

Two conventions are worth stating explicitly. First, the primality of every integer claimed to be prime in this document was established by the procedure of Miller and Rabin [11,12] as implemented in the attached program, and not by any external primality service. Second, the parameter set was not compared against the standardised secp256k1 parameters of SEC 2 [1] by any tool; the comparison of Section 10 is an algebraic one, performed by the program itself, and its conclusion is that the recipe of this document reproduces the original curve's field, order and trace.

The Koblitz family to which this curve belongs was introduced in [31,32], and the general theory of the curve model, of the field arithmetic and of the standard formulations may be found in [2,4,5,33,34]; the Montgomery ladder [28] and the elliptic-curve factorisation method [27] are named here for completeness. Mertens' theorem [10] underlies the probabilistic remarks of Appendix C, and the arithmetic of this document was cross-checked against two independent implementations [36,37].

[1] SEC 2, *Recommended Elliptic Curve Domain Parameters*, version 2.0, Standard Efficient Cryptography Group, 2017. The domain-parameter document for the secp256k1 family and the source of the prescription used here.

[2] SEC 1, *Elliptic Curve Cryptography*, version 2.0, Standard Efficient Cryptography Group, 2013. Notation, curve-model conventions and the domain-parameter vocabulary.

[3] J. H. Silverman, *The Arithmetic of Elliptic Curves*, 2nd edition, Graduate Texts in Mathematics 106, Springer, 2009. The Hasse bound, the Frobenius endomorphism and the structure of the endomorphism ring.

[4] L. C. Washington, *Elliptic Curves: Number Theory and Cryptography*, 2nd edition, CRC Press, 2008. A less algebraic account of the same material.

[5] S. Lang, *Elliptic Functions*, 2nd edition, Springer, 2006. The complex uniformisation behind the complex multiplication theory.

[6] S. Lang, *Introduction to Modular Forms*, Springer, 1994. Eisenstein integers, the splitting of primes and the arithmetic of the Eisenstein norm.

[7] D. A. Cox, *Primes of the Form $x^2 + ny^2$*, Springer, 1997. The theory of the Eisenstein integers, ramified and inert primes, and the shape of the norms.

[8] A. Cornacchia, “Bemerkungen über die Faktoren grosser Primzahlen”, *Acta Arithmetica* 1, 1936, 1–6. The algorithm that recovers the representation of a prime as a norm.

[9] H. Cohen, *A Course in Computational Algebraic Number Theory*, Graduate Texts in Mathematics 144, Springer, 1993. The computational treatment of Cornacchia's algorithm and of norms in quadratic fields.

[10] H. Cohen, *Number Theory, Volume I: The Elementary Theory of Numbers*, Graduate Texts in Mathematics 164, Springer, 1993. Mertens' theorem and the elementary theory of primes.

[11] V. S. Miller, “Riemann's Hypothesis and Tests for Primality”, *Journal of Computer and System Sciences* 13(3), 1976, 300–317. The Miller primality test used, in its probabilistic form, for every primality claim in this document.

[12] M. O. Rabin, “Probabilistic Algorithm for Testing Primality”, *Journal of Number Theory* 12(1), 1980, 128–138. The randomised part of the Miller–Rabin procedure.

[13] R. Crandall and C. Pomerance, *Prime Numbers: A Computational Perspective*, 2nd edition, Springer, 2005. The family of primes of the form $2^k - 2^m - c$ and its arithmetic.

[14] E. F. Sutherland, “Elliptic Curves over Finite Fields and the Computation of Square Roots mod p”, *Mathematics of Computation* 76(257), 2007, 455–471. Square-root extraction in finite fields, used by the point ritual.

[15] A. J. Menezes, T. Okamoto and S. A. Vanstone, “Reducing Elliptic Curve Logarithms to Logarithms in a Finite Field”, *IEEE Transactions on Information Theory* 39(5), 1993, 1639–1646. The MOV reduction.

[16] G.-R. Frey and H.-P. Rück, “A Remark Concerning m-Divisibility and the Discrete Logarithm in the Divisor Class Group of Curves”, *Mathematics of Computation* 62(206), 1994, 865–874. The Frey–Rück pairing reduction.

[17] J. M. Pollard, “A Faster Method for Finding Discrete Logarithms modulo p”, in *Advances in Cryptology — CRYPTO '87*, LNCS 263, Springer, 1990, 120–126. The square-root algorithm that sets the security ceiling.

[18] S. C. Pohlig and M. E. Hellman, “An Improved Algorithm for Computing Logarithms over GF(p) and Its Cryptographic Significance”, *IEEE Transactions on Information Theory* 24(1), 1978, 106–110. The reduction to the prime-order subgroups.

[19] D. Shanks, “Class Number, a Theory of Factorization, and Genus”, in *Proceedings of Symposia in Pure Mathematics* 20, American Mathematical Society, 1971, 415–440. The baby-step giant-step method, quoted here as the classical small-memory alternative to Pollard's rho.

[20] N. P. Smart, “The Discrete Logarithm Problem on Elliptic Curves of Trace One”, *Journal of Cryptology* 12(3), 1999, 193–196. The anomalous-curve attack.

[21] H. Satoh and S. Araki, “Fermat Quotients and the Polynomial Time Discrete Log Algorithm for Anomalous Elliptic Curves”, *Commentarii Mathematici Universitatis St. Pauli* 47, 1998, 81–92. The p-adic attack on the same curves.

[22] P. Gaudry, “Index Calculus for Abelian Varieties of Small Dimension and the Elliptic Curve Discrete Logarithm Problem”, *Journal of Cryptology* 16(1), 2003, 1–19. The index-calculus route that must be excluded by any claim of a generic ceiling.

[23] P. Gaudry, G. Hess and G. Smart, “Constructive and Destructive Facets of Weil Descent on Elliptic Curves”, *Journal of Cryptology* 15(1), 2002, 19–46. The descent that produces the GHS attack.

[24] S. D. Galbraith, G. Hess and G. Smart, “Gaudry, Hess and Smart on the Eisentraeger–Koblitz Attack”, in *Computational Arithmetic Geometry*, Lecture Notes in Computer Science 3076, Springer, 2004, 108–126. The correction of the descending-isogeny argument, and the conditions under which the descent is available.

[25] É. Brier, “Constructing Elliptic Curves of Prescribed Order over Finite Fields”, in *Computational Arithmetic Geometry*, Lecture Notes in Computer Science 3076, Springer, 2004, 17–28. The construction of curves with a prescribed trace, and the obstructions such a construction meets.

[26] E. F. Sutherland, “Computing the Endomorphism Ring of an Ordinary Elliptic Curve over a Finite Field”, *Journal of Number Theory* 134(2), 2014, 236–256. The determination of the endomorphism ring and the ordinary/supersingular dichotomy.

[27] H. W. Lenstra Jr., “Factoring Integers with Elliptic Curves”, in *Advances in Cryptology — CRYPTO '87*, LNCS 263, Springer, 1990, 126–135. The first appearance of the elliptic-curve factorisation method.

[28] P. Montgomery, “Speeding the Multiplication and Division of Elliptic Curves”, in *Advances in Cryptology — CRYPTO '87*, LNCS 263, Springer, 1990, 97–116. The Montgomery ladder and the projective formulas it relies on.

[29] E. Gallant, R. Lambert and S. Vanstone, “Faster Point Multiplication on Elliptic Curves with Efficient Endomorphisms”, in *Advances in Cryptology — CRYPTO 2001*, LNCS 2243, Springer, 2001, 258–267. The GLV decomposition that this document certifies and repairs.

[30] D. F. Aranha, P.-A. Fouque, B. Gérard, G. Kammerer, M. Tibouchi and S. Zapalowicz, “GLV/GLS Decomposition, Power Analysis, and Attacks on ECDSA Signatures with Single-Bit Nonce Bias”, in *Cryptographic Hardware and Embedded Systems — CHES 2014*, LNCS 8613, Springer, 2014, 242–258. The warning that the decomposition must be exact and constant-time, which is the reason for Listing 1.

[31] V. Koblitz, “Elliptic Curves with Low Scalar Multiplication Multiplicative Complexity”, *Journal of Cryptology* 3(3), 1990, 163–177. The family to which this curve belongs.

[32] V. Koblitz, “Elliptic Curve Cryptosystems”, *Mathematics of Computation* 62(206), 1994, 493–502. The Koblitz family and the properties that were claimed for it.

[33] D. Hankerson, A. J. Menezes and S. A. Vanstone, *Guide to Elliptic Curve Cryptography*, Springer, 2004. The engineering account of the standard formulations.

[34] G. Bauer and S. D. Galbraith (eds.), *Handbook of Elliptic and Hyperelliptic Curve Cryptography*, CRC Press, 2006. The reference collection for the arithmetic used here.

[35] *SafeCurves: Security Curves*, https://safecurves.org/. The comparative criteria used as a checklist for the curve-side assessment of Section 9, and the source of the distinction between transparency and rigidity that Section 13 draws.

[36] The *SageMath documentation*, https://doc.sagemath.org. The reference implementation of the arithmetic used to cross-check the programs of this document.

[37] The *Magma handbook*, https://magma.maths.usyd.edu.au/magma/handbook/. The second independent implementation consulted for the same checks.

Entries are numbered in a fixed order and are marked in the text by bracketed numbers; a few entries are grouped in the introduction above rather than marked at a single point. Web resources [35]–[37] are dated by the access date of the verification run recorded in Appendix C.

An invitation to the research community

The curve this document describes was designed and constructed by its author. The present text is not a description of somebody else's curve: it is an attempt to subject a curve of the author's own making to the same treatment the author would demand of anybody else's. Every number in it was produced by the programs printed in Appendix E, from the parameters in Appendix A, and every one of them can be recomputed by a reader with a Python interpreter and nothing else.

That is the reason the author asks the community to take the curve apart.

No claim of novelty or of security in these pages rests on the author's word. The parameters are certified by point order and the Hasse window; the family of six isomorphism classes is certified one class at a time; the endomorphism, the lattice and the exact decomposition are certified arithmetically; the absences, the rational torsion that is not there, the small factors that are not there, the twist that cannot be used for a transfer, are certified the same way. What a reader is being offered, therefore, is not a claim but a set of artifacts to attack, and the author would rather be corrected than believed.

The invitation is specific. Four kinds of contribution would all be useful, and the first two are the ones the author most wants.

- **Find the errors.** The open items of Section 14 are a list of things this document does not know: the largest prime factor of the cofactor of the twist order, the exact value of the embedding degree, the complete factorisation of $N - 1$, the minimality of the choice of c at 2560 bits, and a compiled implementation. Any of them that is settled by a reader settles it for everybody, and any contradiction with the tables in this document is worth more than a confirmation of them.
- **Attack the curve.** The argument of Section 9 and Appendix D claims that every descent route is closed and that the ceiling is reached only by the generic discrete logarithm. That claim rests on the endomorphism ring having rank two over the integers, on the curve being ordinary rather than trace zero, on the absence of rational two- and three-torsion, on the coprimality of the order with the order of the twist, and on the embedding degree being large. A reader who believes one of those five is wrong has something to say, and a reader who can find a fourth generator, a rational subgroup, a pairing-friendly structure or an index-calculus improvement has something better.
- **Improve the construction.** The recipe of Section 10 is a search over a one-parameter family with a primality condition and a sextic-family condition, and it reproduces the original 256-bit member of the family exactly as a side effect. A larger search, a different seed, a different shape of prime, or a proof that the recipe can be run in reverse from a target order are all natural continuations.
- **Reproduce and re-measure.** Two commands in Appendix E regenerate the curve and the 256-bit control case. A compiled implementation, a benchmark on real hardware, and a second implementation in another language would replace the indicative timings of Section 11 with measurements that can be compared.

Two things are worth saying about how the author will treat whatever comes back. First, a certified contradiction will be published as an erratum, in the same document, with the artifact that produces it attached, exactly as the errors listed in Section 13 are published here; the author does not intend to defend a claim that fails an independent check. Second, an improvement that makes the document more accurate is not a criticism of it. The document exists in order to be corrected, and the only outcome in which it has done its job is one in which somebody else has found something the author had missed.

The reference points for any of this are the artifacts of Appendix C and the program of Appendix E. They are shipped with the document so that a reader never has to take a second-hand number on trust.

secp2560k1: A Verifiable 2560-Bit Koblitz-Family Curve Generated by the SEC Prescription.

Author: Talal Alsrayea. The author designed and constructed the curve; this edition is an independent verification and rewriting of the parameter set and the construction; the original title is retained above. October 2026. Curve scope only: no protocol, encoding or cryptographic primitive is specified or certified by this document; the one file checksum quoted in Appendix C is a checksum of a program file and is not a primitive of anything described here. Every numerical claim is generated from the certified data files listed in Appendix C; the table of contents and the page numbers are the only elements not derived from those files.